

Manual Profesional de Metanálisis en R

Basado en el blog de Dan Quintana

Tabla de Contenidos

1. 1. Three common mistakes in meta-analysis
2. 2. How to identify and deal with outliers in meta-analysis
3. 3. How to perform a Bayesian meta-analysis in R
4. 4. Introducing the {metameta} R package
5. 5. Equivalence-enhanced forest plots with power tiles for meta-analysis
6. 6. Sunset funnel plots for meta-analysis
7. 7. Oh my GOSH: Calculating all possible meta-analysis study combinations
8. 8. How to make a free website in R
9. 9. Labeling boxplot outliers in R
10. 10. Equivalence testing: What to do with a non-significant result
11. 11. Reproducible and future-proof analysis in R
12. 12. How to visualise research trends in R

Three common mistakes in meta-analysis

Fuente: <https://www.dsquintana.blog/three-common-mistakes-in-meta-analysis-publication-bias-funnel-plot/>

Meta-analysis has become a popular tool to summarizing data from a body of work investigating a common research question. As this provides a putatively objective means of summarizing a body of data, it has become the gold standard for evidence when developing health guidelines.

However, there are several issues in meta-analysis that can contribute to inaccuracies. Due to these limitations, meta-analysis—or “mega-silliness”, to borrow a term from an early detractor—has been criticized for almost as long as the tool has been available.

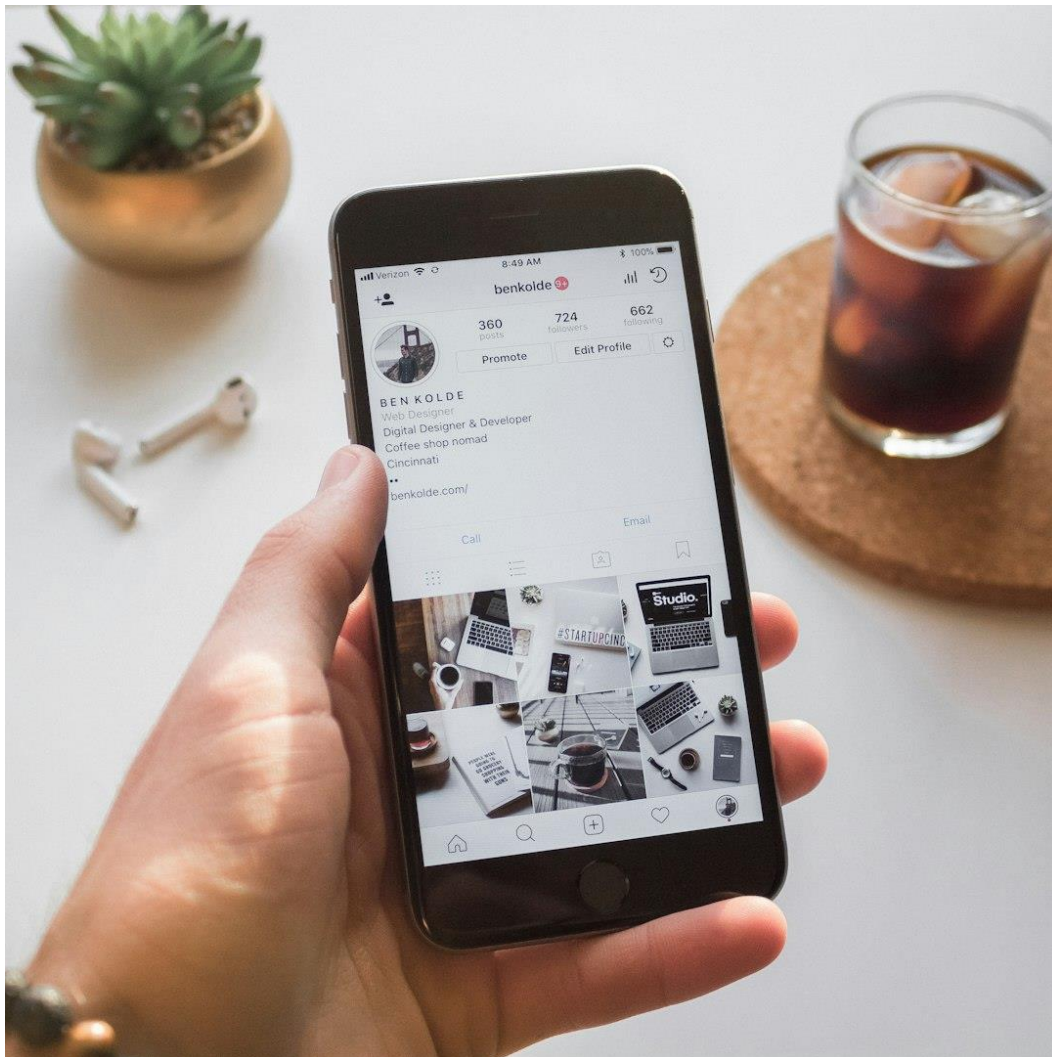
There are several means to help ensure meta-analysis quality, which will be outlined later in this post. But first, let’s consider this (fictitious) extract from the results section of a meta-analysis that I will use to briefly illustrate three common issues in meta-analysis:

“As hypothesized, a meta-analysis synthesizing eight effect sizes from seven studies suggests that social media usage is positively associated with self-reported levels of anxiety [$r = .073$, 95% CI (.004, 0.142); $p = .038$; Fig. 1A]. Egger’s regression test suggested no evidence of publication bias ($p = .19$; Fig. 1B).”

When this imaginary author—who also happens to be releasing a book on the dangers of social media—initially searched for studies, they actually found nine studies fulfilling their inclusion criteria. However, the summary effect size of this meta-analysis of nine studies [$r = .062$, 95% CI (- .004, 0.127)] was associated with a p-value of .061 (Figure 2).

As this outcome would hinder the chances of publication (and book sales), they took a closer look at study six, which reported a negative association between social media usage and anxiety levels. This study happened to be the earliest study included in the meta-analysis.

Study six did not include Instagram usage (which became popular during 2012), so they adjusted the eligibility criteria to only include studies published from 2013 onwards, justified by a (post hoc) hypothesis that Instagram usage is most associated with anxiety.



With the removal of the study, the meta-analysis became statistically significant. While this fabricated scenario might seem extreme at first glance, almost 40% of over 2,000 surveyed psychologists admitted that they had excluded data after looking at the impact of doing so.

Meta-analysis pre-registration provides a remedy to analytic flexibility as it provides an a priori record of the researcher's analysis intentions. In the above example, exclusion criteria would have been specified before the analysis was conducted.

A common critique of pre-registered analyses is that they offer no flexibility for analysis. However, this is incorrect as pre-registration does not lock the researcher into a specific plan but instead helps clearly delineate pre-planned and exploratory analysis.

In the above case, the researcher would be free to report their analysis with the removal of the study, but they would have to state their reasoning, which is flimsy at best. For guidance, the Preferred Reporting Items for Systematic Reviews and Meta-Analysis protocols (PRISMA-P) document provides a list of items that should be addressed in a meta-analysis protocol.

There are several options available for meta-analysis pre-registration. First, a time-stamped meta-analysis protocol following PRISMA-P guidelines can be uploaded to Open Science Framework.

Second, if the meta-analysis investigates a health-related outcome, it can be pre-registered at PROSPERO. The PROSPERO website guides the researcher through a step-by-step registration form, which is used to generate a pre-registration record. These records can be amending, but any changes are timestamped.

Third, a meta-analysis protocol can be submitted for peer-review at a number of journals, such as Systematic Reviews. While the peer-review process obviously takes more time than uploading a protocol, it can provide important feedback that can improve the meta-analysis.

Fourth, a meta-analysis can be submitted using the Registered Report format, in which the meta-analysis is reviewed in two stages. In the first stage, the meta-analysis introduction and methods sections are peer-reviewed before data collection. If the study rationale and methodological approach is deemed appropriate, the journal will provide provisional acceptance of the final paper.



Once the data is collected and analyzed, the paper is submitted again for the second stage review which includes with the same introduction and methods as stage one, but with the addition of results and conclusions. In the second stage of review, the focus is on whether the results and interpretation are in line with the pre-approved method.

Importantly, paper acceptance is not contingent on the statistical significance of results. As of August 2018, there are 42 journals that offer Registered Report meta-analysis, with the majority of these journals publishing research from the biobehavioral sciences.

Publication bias is a well-recognized issue in meta-analysis. As statistically significant studies are more likely to be published, it is likely that there are several non-significant studies left languishing in researcher's file-drawers for any given research area, which cannot contribute to meta-analyses.

Thus, a meta-analysis summary effect size may not best represent the "true" effect size. A common approach to assess the risk of publication is to construct a funnel plot, which plots effect sizes against a measure of variance (e.g., standard errors). A symmetrical funnel plot is indicative of low risk of publication bias. Despite the inherent subjectivity of visually

inspecting a funnel plot to assess evidence of asymmetry, many researchers still rely on this tool.

Now let's return to our imaginary study. As our fictitious researcher was aware of issues with subjective funnel plot assessments, they objectively assessed asymmetry by constructing a funnel plot. As researchers are poor at identifying asymmetry from funnel plots, they followed recommendations and performed Egger's regression test to complement the funnel plot (Fig. 1A).

A statistically significant test is indicative of funnel plot asymmetry. In this case, the test was not statistically significant ($p = .19$). Thus, our researcher concluded that there was no risk of publication bias. A closer look at the data, however, reveals that this conclusion was most likely incorrect.

A common misunderstanding of funnel plots and their associated tests is that they measure publication bias, when in fact they measure small study bias. This type of bias might include publication bias, but it also encompasses other types of bias commonly associated with small studies, such as study quality.

A relatively simple funnel plot modification, called a contour-enhanced funnel plot, can help better understand the risk of publication bias compared to small study bias. For a given range of effect sizes and variances, it is possible to calculate levels of statistical significance at every point. As a result, it is possible to visualise ranges of statistical significance (e.g., $p = .05$ to $.01$) on a funnel plot.

Comparing our original funnel plot (Fig. 3A) with a contour-enhanced funnel plot (Fig. 3B) reveals that despite no apparent evidence of funnel plot asymmetry, half of the included effect sizes had p-values were very close to .05. It is highly improbable that half of the available p-values would lie so close to .05. Either these significant studies were also the result of selective analyses or there are several unpublished studies that exist.

Several measures have developed to correct for bias in meta-analysis, such as p-curve, p-uniform, and the three-parameter selection model, with the latter demonstrating superior performance. But without access unpublished research these methods will always be estimates, at best. One plausible solution to this scenario is to conduct a meta-analysis that only includes pre-registered empirical studies, such as the recent meta-analysis on "power posing" from Gronau and colleagues.

Statistically dependent effect sizes

Now to our final common issue in meta-analysis: effect size dependence. Notice that in the earlier meta-analysis example there were eight effect sizes but only seven studies? This was due to two effect sizes being extracted from study four (Fig. 1A). This is a common occurrence as papers sometimes report several outcome measures.

However, as these effect sizes were derived from the same study population they are statistically dependent. As typical meta-analysis procedures assume that effect sizes are statistically independent, combining dependent meta-analyses can lead to biased results.

There are several approaches to for addressing statistical dependency in meta-analysis. First, you can simply choose one effect size per study. While straightforward, a pre-specified system is required to help select the chosen effect sizes, otherwise it can be tempting to simply choose the largest effect. What's more, this approach excludes potentially valuable effect sizes from your analysis. To include more than one effect from a study, it is possible to statistically aggregate effects. But to do this, you need to include a measure of within-study correlation, which is almost never reported in papers.

If the paper contains raw data, an occurrence which is almost as rare, within-study correlations can be calculated. Therefore, researchers almost always need to estimate within-study correlations if this approach is to be taken. Alternatively, robust variance estimation can be used to account for effect size dependency without a need to know within-study correlations.

Meta-analysis is an important tool for better understanding often disparate research areas. This method can provide a more objective summary of a research area than simply "vote counting" the number of significant and non-significant results.

But like almost any statistical tool, meta-analysis can facilitate inaccurate inferences if used inappropriately. With this post, I hope to have provided a brief overview highlighting some common issues in meta-analysis and practical solutions to these problems.

This is an edited version of an article published in *Psykologisk Tidsskrift NTNU*, a psychology magazine run by students at the Norwegian University of Science and Technology. Here is a citable preprint of the article and the R script and data to perform the analyses described in this post are available [here](#). My thanks to the Editor of *Psykologisk Tidsskrift NTNU* for kindly providing permission to repost this article as a blog post.

How to identify and deal with outliers in meta-analysis

Fuente: <https://www.dsquintana.blog/how-do-you-decide-which-studies-in-a-meta-analysis-are-influential-and-should-be-removed/>

One of the first steps when analysing primary data is to visualise your results. This can help identify unusual trends, outliers, and data points that have a disproportionate influence on your results, which might not be clear by looking at summary statistics alone.

To demonstrate, let's have a look at a set of thirteen datasets from the `datasaurus` R package, which each contain x-values and y-values that all generate the same means and standard deviations, and roughly the same correlation coefficients.

```
datasaurus

install.packages("datasauRus")
library(datasauRus)
library(viridis)
if(requireNamespace("dplyr")) {
  suppressPackageStartupMessages(library(dplyr))
  datasaurus_dozen %>%
    group_by(dataset) %>%
    summarize(
      mean_x      = mean(x),
      mean_y      = mean(y),
      std_dev_x   = sd(x),
      std_dev_y   = sd(y),
      corr_x_y    = cor(x, y)
    )
}

install.packages("datasauRus")
library(datasauRus)
library(viridis)
if(requireNamespace("dplyr")) {
  suppressPackageStartupMessages(library(dplyr))
  datasaurus_dozen %>%
    group_by(dataset) %>%
    summarize(
      mean_x      = mean(x),
      mean_y      = mean(y),
      std_dev_x   = sd(x),
      std_dev_y   = sd(y),
      corr_x_y    = cor(x, y)
    )
}
```

This will code generate the following output:

```
# A tibble: 13 x 6
  dataset    mean_x mean_y std_dev_x std_dev_y corr_x_y
  <chr>      <dbl> <dbl>    <dbl>    <dbl>    <dbl>
1 away      54.3  47.8     16.8     26.9   -0.0641
2 bullseye  54.3  47.8     16.8     26.9   -0.0686
```

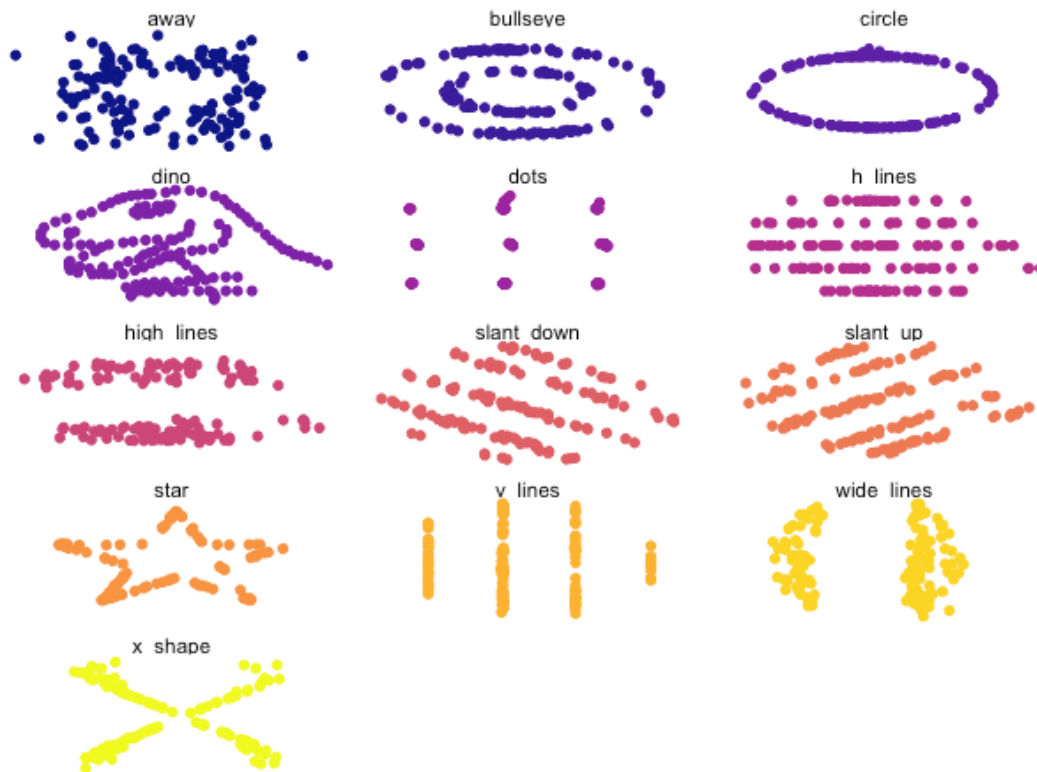

3	circle	54.3	47.8	16.8	26.9	-0.0683
4	dino	54.3	47.8	16.8	26.9	-0.0645
5	dots	54.3	47.8	16.8	26.9	-0.0603
6	h_lines	54.3	47.8	16.8	26.9	-0.0617
7	high_lines	54.3	47.8	16.8	26.9	-0.0685
8	slant_down	54.3	47.8	16.8	26.9	-0.0690
9	slant_up	54.3	47.8	16.8	26.9	-0.0686
10	star	54.3	47.8	16.8	26.9	-0.0630
11	v_lines	54.3	47.8	16.8	26.9	-0.0694
12	wide_lines	54.3	47.8	16.8	26.9	-0.0666
13	x_shape	54.3	47.8	16.8	26.9	-0.0656

```
# A tibble: 13 x 6
  dataset mean_x mean_y std_dev_x std_dev_y corr_x_y
  <chr>    <dbl> <dbl>    <dbl>    <dbl>    <dbl>
1 away    54.3  47.8    16.8    26.9   -0.0641
2 bullseye 54.3  47.8    16.8    26.9   -0.0686
3 circle   54.3  47.8    16.8    26.9   -0.0683
4 dino     54.3  47.8    16.8    26.9   -0.0645
5 dots     54.3  47.8    16.8    26.9   -0.0603
6 h_lines  54.3  47.8    16.8    26.9   -0.0617
7 high_lines 54.3  47.8    16.8    26.9   -0.0685
8 slant_down 54.3  47.8    16.8    26.9   -0.0690
9 slant_up  54.3  47.8    16.8    26.9   -0.0686
10 star    54.3  47.8    16.8    26.9   -0.0630
11 v_lines  54.3  47.8    16.8    26.9   -0.0694
12 wide_lines 54.3  47.8    16.8    26.9   -0.0666
13 x_shape  54.3  47.8    16.8    26.9   -0.0656
```

Next, let's visualise these x-y relationships via a series of scatterplots:

```
if(requireNamespace("ggplot2")){
  library(ggplot2)
  library(viridis)
  dp <- ggplot(datasaurus_dozen, aes(x=x, y=y, colour=dataset))+
    geom_point()+
    theme_void()+
    theme(legend.position = "none")+
    facet_wrap(~dataset, ncol=3)
  dp + scale_colour_viridis_d(option = "plasma")
}

if(requireNamespace("ggplot2")){
  library(ggplot2)
  library(viridis)
  dp <- ggplot(datasaurus_dozen, aes(x=x, y=y, colour=dataset))+
    geom_point()+
    theme_void()+
    theme(legend.position = "none")+
    facet_wrap(~dataset, ncol=3)
  dp + scale_colour_viridis_d(option = "plasma")
}
```



Despite having similar statistical characteristics, the shape of the data was different between datasets (and now you probably understand why this package is called "datasaurus", if you look carefully).

Meta-analysis is no different. Typically, a meta-analyst will construct a forest plot to visualise effect sizes and their variances, which are used to synthesize data into a summary effect size. Here's an example meta-analysis and forest plot and the code to construct it, using a dataset from metafor and a meta-analysis and forest plot function from the meta package.

```
metafor

meta

library(meta)
library(metafor)

dat <- dat.molloy2014

datcor <- metacor(ri,
  ni,
  data = dat,
  studlab = paste(authors),
  method.tau = "REML",
  comb.random = TRUE,
  comb.fixed = FALSE,
  sm = "ZCOR")
```

```

meta::forest(datcor, print.I2 = FALSE)

datcor

## OUTPUT WITH SOME INFO OMITTED ##

Number of studies combined: k = 16

COR          95%-CI      z  p-value
Random effects model 0.1488 [0.0878; 0.2087] 4.75 < 0.0001

Quantifying heterogeneity:
tau^2 = 0.0081 [0.0017; 0.0378]; tau = 0.0901 [0.0412; 0.1944]
I^2 = 60.7% [32.1%; 77.2%]; H = 1.59 [1.21; 2.10]

Test of heterogeneity:
  Q d.f. p-value
38.16  15  0.0009

library(meta)
library(metafor)

dat <- dat.molloy2014

datcor <- metacor(ri,
                  ni,
                  data = dat,
                  studlab = paste(authors),
                  method.tau = "REML",
                  comb.random = TRUE,
                  comb.fixed = FALSE,
                  sm = "ZCOR")

meta::forest(datcor, print.I2 = FALSE)

datcor

## OUTPUT WITH SOME INFO OMITTED ##

Number of studies combined: k = 16

COR          95%-CI      z  p-value
Random effects model 0.1488 [0.0878; 0.2087] 4.75 < 0.0001

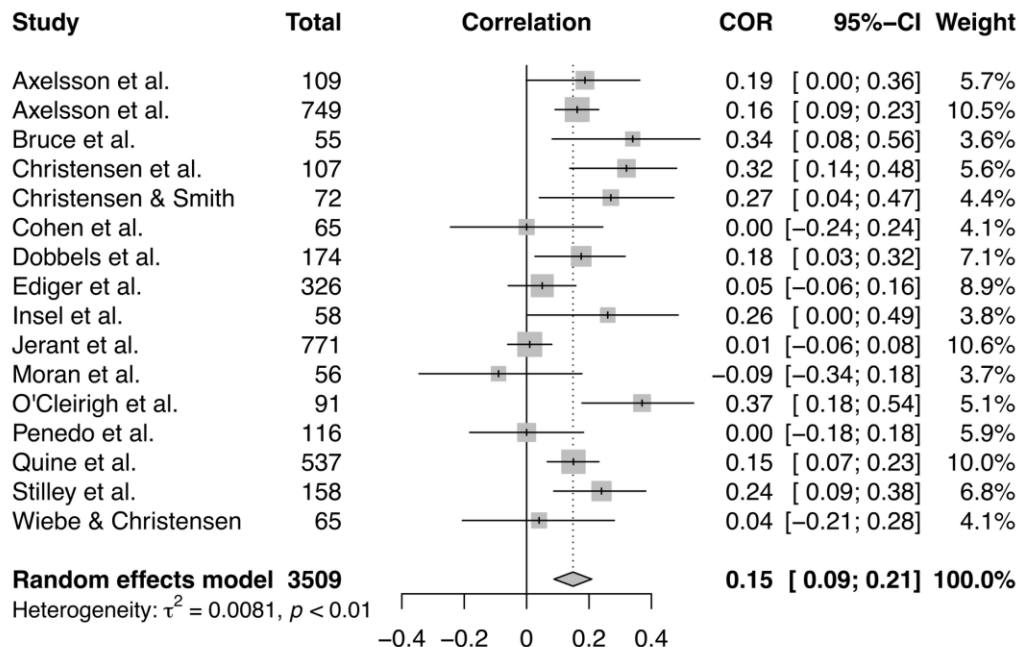
Quantifying heterogeneity:
tau^2 = 0.0081 [0.0017; 0.0378]; tau = 0.0901 [0.0412; 0.1944]
I^2 = 60.7% [32.1%; 77.2%]; H = 1.59 [1.21; 2.10]

Test of heterogeneity:
  Q d.f. p-value
38.16  15  0.0009

```

Looking at the results of the meta-analysis, the random-effects model was statistically significant ($p < .0001$), with a summary effect size estimate of 0.1488. The test of heterogeneity was also statistically significant ($p < .0009$).

Now let's look at our forest plot. Of course, this is very subjective, but forest plots can identify studies that are worth closer inspection.



On first glance, there aren't any studies that stand out in this meta-analysis, but it's always worth taking a more in-depth look at possible outliers and influential studies.

One approach for outlier detection is a "leave-one-out" analysis, which assesses the influence of individual studies by performing a series of meta-analyses that leave out one of the studies in the original meta-analysis. The general concept with this analysis is that by removing one study, you can observe how much the results change, thus demonstrating the effect of the inclusion of that particular study on your results.

It's worth noting that none of these tests within a leave-one-out analysis is a definitive test, as you're arbitrarily excluding one study per analysis that would ordinarily meet your inclusion criteria. So if your primary meta-analysis was not statistically significant but one of your leave-one-out meta-analyses were significant, you can't make big claims regarding the significance of your finding. Instead, consider this approach a sensitivity test.

Leave-one-out analysis can also help identify sources of heterogeneity. Perhaps there are two studies that seem to be influential—what is it about those studies that make them influential? Maybe they include participants from the same kind of population? Sometimes these factors don't become clear until you see these studies grouped together via leave-one-out analysis.

It's straightforward to perform a leave-one-out analysis using the metafor R package. The `influence()` function contains a suite of leave-one-out diagnostic tests, summarised in Viechtbauer & Cheung (2010), that you can run to help identify influential studies.

```
metafor

influence()

library(metafor)

dat <- dat.molloy2014

dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat.molloy2014,
              slab=paste(authors, year, sep=", "))

res <- rma(yi, vi, data=dat)
inf <- influence(res)
inf

# OUTPUT #
```

	rstudent	dffits	cook.d	cov.r
Axelsson et al., 2009	0.2918	0.0485	0.0025	1.1331
Axelsson et al., 2011	0.1196	-0.0031	0.0000	1.2595
Bruce et al., 2010	1.2740	0.2595	0.0660	0.9942
Christensen et al., 1999	1.4711	0.3946	0.1439	0.9544
Christensen & Smith, 1995	0.8622	0.1838	0.0339	1.0505
Cohen et al., 2004	-0.9795	-0.2121	0.0455	1.0639
Dobbels et al., 2005	0.2177	0.0296	0.0010	1.1740
Ediger et al., 2007	-0.9774	-0.3120	0.1001	1.1215
Insel et al., 2006	0.7264	0.1392	0.0195	1.0561
Jerant et al., 2011	-1.8667	-0.5861	0.2198	0.8502
Moran et al., 1997	-1.4985	-0.2771	0.0756	1.0073
O'Cleirigh et al., 2007	1.8776	0.4918	0.2148	0.8819
Penedo et al., 2003	-1.1892	-0.2939	0.0859	1.0550
Quine et al., 2012	-0.0020	-0.0423	0.0021	1.2524
Stilley et al., 2004	0.8066	0.2126	0.0459	1.0907
Wiebe & Christensen, 1997	-0.7160	-0.1656	0.0280	1.0853
	tau2.del	QE.del	hat	weight
Axelsson et al., 2009	0.0091	37.7109	0.0568	5.6776
Axelsson et al., 2011	0.0100	36.7672	0.1054	10.5396
Bruce et al., 2010	0.0075	35.3930	0.0364	3.6432
Christensen et al., 1999	0.0068	33.5886	0.0562	5.6195
Christensen & Smith, 1995	0.0082	36.5396	0.0441	4.4069
Cohen et al., 2004	0.0084	37.1703	0.0411	4.1094
Dobbels et al., 2005	0.0094	37.6797	0.0714	7.1362
Ediger et al., 2007	0.0084	36.1484	0.0889	8.8886
Insel et al., 2006	0.0083	37.0495	0.0379	3.7886
Jerant et al., 2011	0.0047	25.0661	0.1058	10.5826
Moran et al., 1997	0.0077	35.6617	0.0369	3.6922
O'Cleirigh et al., 2007	0.0059	31.9021	0.0511	5.1150
Penedo et al., 2003	0.0080	36.3291	0.0587	5.8732
Quine et al., 2012	0.0100	37.7339	0.0998	9.9778

```

Stilley et al., 2004      0.0083 35.8385 0.0684  6.8403
Wiebe & Christensen, 1997 0.0087 37.7017 0.0411  4.1094
                        dfbs inf
Axelsson et al., 2009    0.0481
Axelsson et al., 2011   -0.0032
Bruce et al., 2010      0.2623
Christensen et al., 1999 0.3994
Christensen & Smith, 1995 0.1837
Cohen et al., 2004      -0.2112
Dobbels et al., 2005    0.0296
Ediger et al., 2007     -0.3128
Insel et al., 2006      0.1387
Jerant et al., 2011     -0.5430
Moran et al., 1997      -0.2791
O'Cleirigh et al., 2007 0.5059
Penedo et al., 2003     -0.2941
Quine et al., 2012      -0.0434
Stilley et al., 2004    0.2125
Wiebe & Christensen, 1997 -0.1642

```

```
library(metafor)
```

```
dat <- dat.molloy2014
```

```

dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat.molloy2014,
              slab=paste(authors, year, sep=", "))

```

```

res <- rma(yi, vi, data=dat)
inf <- influence(res)
inf

```

```
# OUTPUT #
```

```

                        rstudent  dffits  cook.d  cov.r
Axelsson et al., 2009    0.2918  0.0485 0.0025 1.1331
Axelsson et al., 2011    0.1196 -0.0031 0.0000 1.2595
Bruce et al., 2010      1.2740  0.2595 0.0660 0.9942
Christensen et al., 1999 1.4711  0.3946 0.1439 0.9544
Christensen & Smith, 1995 0.8622  0.1838 0.0339 1.0505
Cohen et al., 2004      -0.9795 -0.2121 0.0455 1.0639
Dobbels et al., 2005    0.2177  0.0296 0.0010 1.1740
Ediger et al., 2007     -0.9774 -0.3120 0.1001 1.1215
Insel et al., 2006      0.7264  0.1392 0.0195 1.0561
Jerant et al., 2011     -1.8667 -0.5861 0.2198 0.8502
Moran et al., 1997      -1.4985 -0.2771 0.0756 1.0073
O'Cleirigh et al., 2007 1.8776  0.4918 0.2148 0.8819
Penedo et al., 2003     -1.1892 -0.2939 0.0859 1.0550
Quine et al., 2012      -0.0020 -0.0423 0.0021 1.2524
Stilley et al., 2004    0.8066  0.2126 0.0459 1.0907
Wiebe & Christensen, 1997 -0.7160 -0.1656 0.0280 1.0853
                        tau2.del  QE.del   hat  weight
Axelsson et al., 2009    0.0091 37.7109 0.0568  5.6776
Axelsson et al., 2011    0.0100 36.7672 0.1054 10.5396

```

Bruce et al., 2010	0.0075	35.3930	0.0364	3.6432
Christensen et al., 1999	0.0068	33.5886	0.0562	5.6195
Christensen & Smith, 1995	0.0082	36.5396	0.0441	4.4069
Cohen et al., 2004	0.0084	37.1703	0.0411	4.1094
Dobbels et al., 2005	0.0094	37.6797	0.0714	7.1362
Ediger et al., 2007	0.0084	36.1484	0.0889	8.8886
Insel et al., 2006	0.0083	37.0495	0.0379	3.7886
Jerant et al., 2011	0.0047	25.0661	0.1058	10.5826
Moran et al., 1997	0.0077	35.6617	0.0369	3.6922
O'Cleirigh et al., 2007	0.0059	31.9021	0.0511	5.1150
Penedo et al., 2003	0.0080	36.3291	0.0587	5.8732
Quine et al., 2012	0.0100	37.7339	0.0998	9.9778
Stilley et al., 2004	0.0083	35.8385	0.0684	6.8403
Wiebe & Christensen, 1997	0.0087	37.7017	0.0411	4.1094
	dfbs	inf		
Axelsson et al., 2009	0.0481			
Axelsson et al., 2011	-0.0032			
Bruce et al., 2010	0.2623			
Christensen et al., 1999	0.3994			
Christensen & Smith, 1995	0.1837			
Cohen et al., 2004	-0.2112			
Dobbels et al., 2005	0.0296			
Ediger et al., 2007	-0.3128			
Insel et al., 2006	0.1387			
Jerant et al., 2011	-0.5430			
Moran et al., 1997	-0.2791			
O'Cleirigh et al., 2007	0.5059			
Penedo et al., 2003	-0.2941			
Quine et al., 2012	-0.0434			
Stilley et al., 2004	0.2125			
Wiebe & Christensen, 1997	-0.1642			

This analysis also provides a classification for what's considered influential. Look for an asterisk in the last column of the output, called 'inf' (this particular analysis had no influential studies, so there are no asterisks here). See the metafor documentation for what's considered influential, but here's an important point from this document worth emphasising:

metafor

Note that the chosen cut-offs are (somewhat) arbitrary. Substantively informed judgment should always be used when examining the influence of each study on the results.

Note that the chosen cut-offs are (somewhat) arbitrary. Substantively informed judgment should always be used when examining the influence of each study on the results.

You can also make the same plot (with slightly different names for the influence tests) using the dmetar package:

dmetar

```
library(devtools)
devtools::install_github("MathiasHarrer/dmetar")
```

```
library(dmetar)

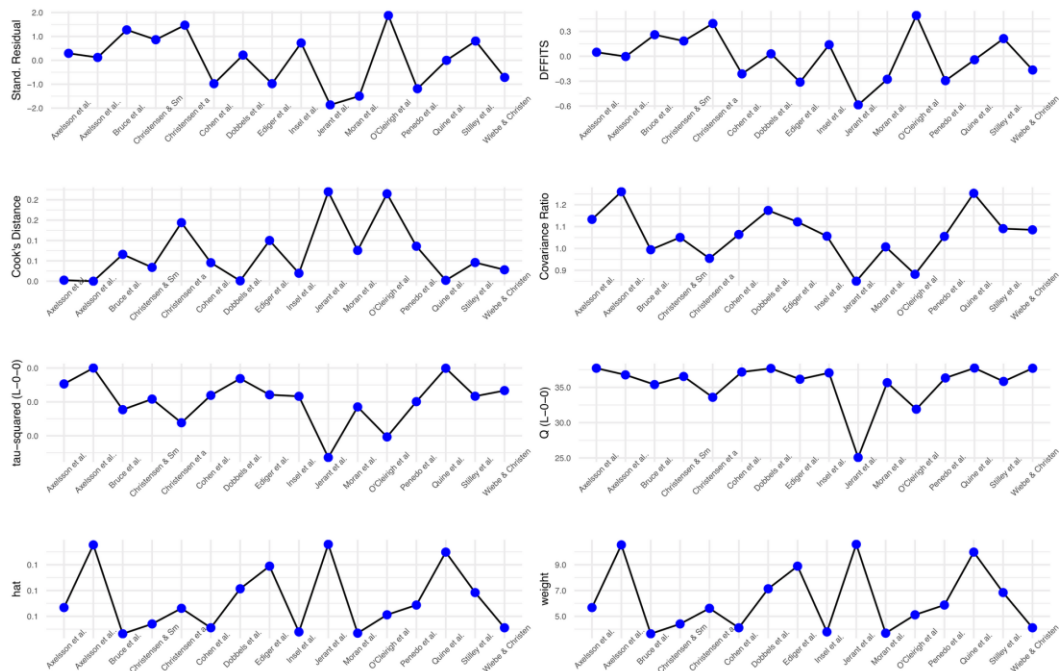
inf.analysis <- InfluenceAnalysis(x = datcor, random = TRUE)

plot(inf.analysis, "influence")

library(devtools)
devtools::install_github("MathiasHarrer/dmetar")
library(dmetar)

inf.analysis <- InfluenceAnalysis(x = datcor, random = TRUE)

plot(inf.analysis, "influence")
```



If you want a nice visualisation for your leave-one-out analysis, you can pair the 'meta' and 'dmetar' packages:

```
library(dmetar)
library(meta)
library(metafor)

dat <- dat.molloy2014

datcor <- metacor(ri,
  ni,
  data = dat,
  studlab = paste(authors),
  method.tau = "REML",
  comb.random = TRUE,
  comb.fixed = FALSE,
  sm = "ZCOR")
```



```

inf.analysis <- InfluenceAnalysis(x = datcor,
                                random = TRUE)

plot(inf.analysis, "es")

library(dmetar)
library(meta)
library(metafor)

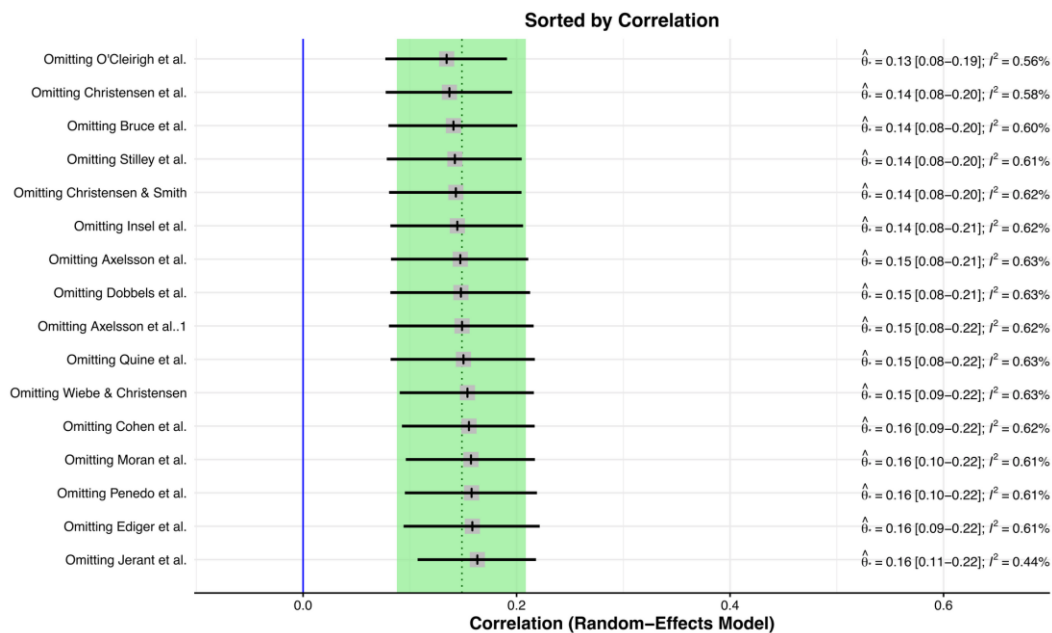
dat <- dat.molloy2014

datcor <- metacor(ri,
                  ni,
                  data = dat,
                  studlab = paste(authors),
                  method.tau = "REML",
                  comb.random = TRUE,
                  comb.fixed = FALSE,
                  sm = "ZCOR")

inf.analysis <- InfluenceAnalysis(x = datcor,
                                random = TRUE)

plot(inf.analysis, "es")

```



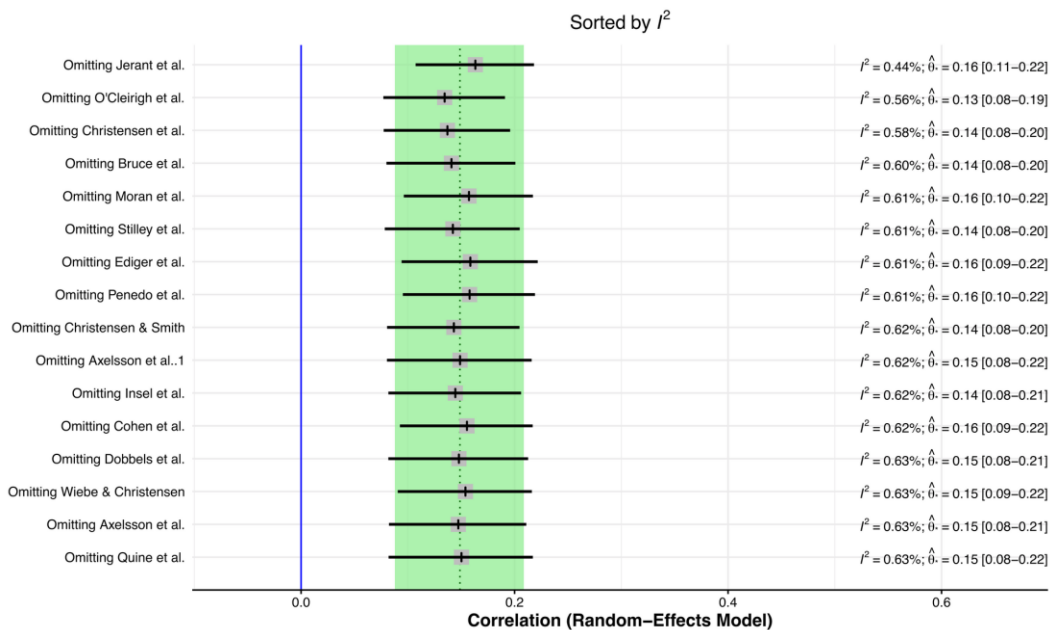
This plot is like a forest plot, but instead of visualising the effect size and variance for each study in each row, it visualises the summary effect sizes for meta-analyses without the study named in each row. The original summary effect size (including all studies) is shown as a dotted vertical line, and the 95% confidence interval of the original meta-analysis is shown by the green bounds. In this example, there isn't a big shift between the smallest correlation at the top and the largest correlation at the bottom.

Outliers just based on effect size are easier to eyeball from a forest plot, however, an outlier of heterogeneity is a little trickier to identify.

To better understand studies that are influential in terms of heterogeneity, you can also order the plot via I^2 , which is one measure of heterogeneity. While a single study effect size may not have a large influence on the summary effect size, it could have an impact on heterogeneity.

```
plot(inf.analysis, "I2")
```

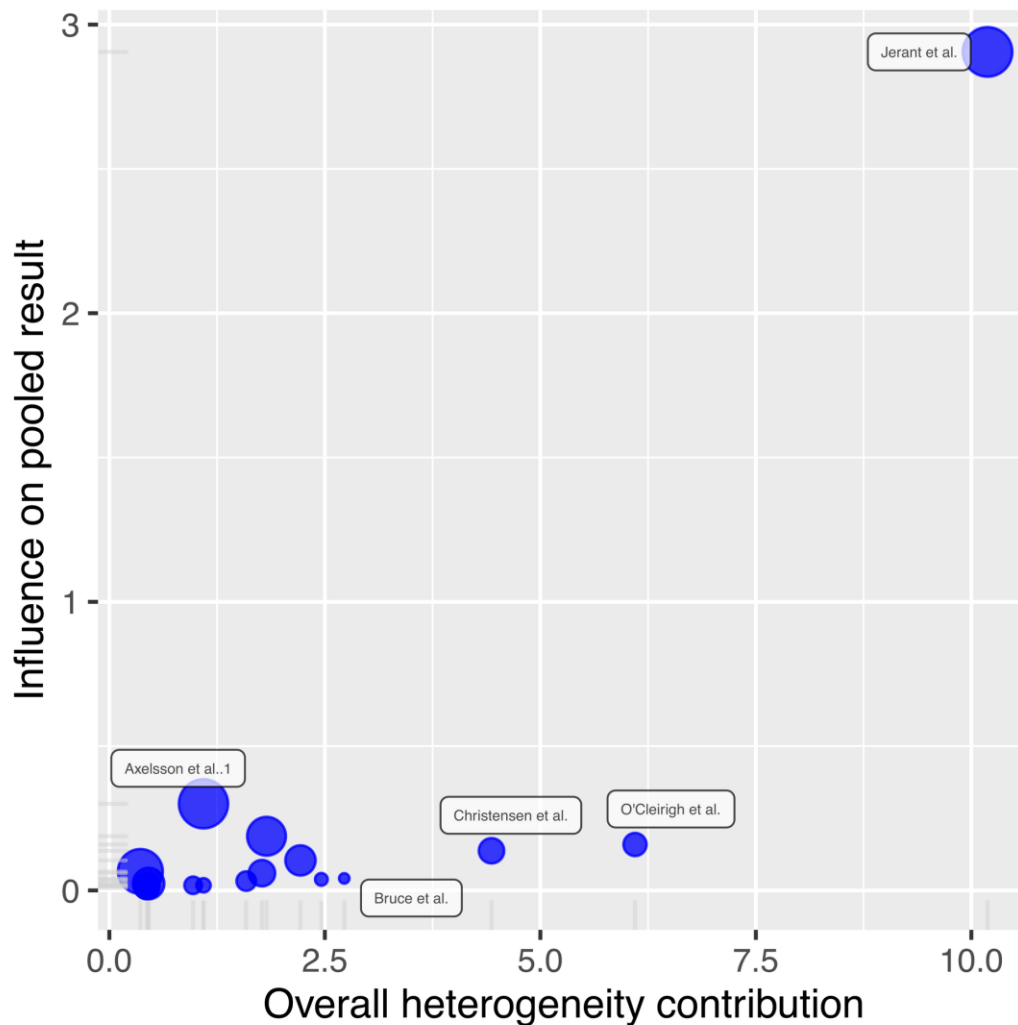
```
plot(inf.analysis, "I2")
```



Removing the Jerant et al study seems to have an appreciable effect on reducing study heterogeneity. To more closely explore this, we can construct a Baujat plot, which shows the overall heterogeneity contribution for each study against the influence on the pooled result for each study.

```
plot(inf.analysis, "baujat")
```

```
plot(inf.analysis, "baujat")
```



Jerant et al is really sticking out now, as it has almost three times more influence on the pooled result compared to all other studies, and a comparatively high overall heterogeneity contribution. If we look back at our leave-one-out plot ordered by correlation, removing Jerant et al leads to the largest effect size, but it's not the largest by much. This indicates that this Jerant et al is pulling down the overall summary effect size estimate, but not by a large amount.

What do you do if your analysis identifies potential outlier studies?

I don't think that outliers shouldn't be removed from your primary analysis, unless they're implausible.

What do I mean by implausible?

To give a recent example, a man in the UK was put into the priority list for a COVID vaccine due to morbid obesity. Thinking that this may have been a mistake, as he considered his weight to be relatively normal, he followed this up to discover that the local authorities had him registered with a BMI of 28,000, because his height of 6ft 2ins was noted as 6.2cm

instead. As BMI of 40 and over is considered 'morbidly obese', a BMI value of 28,000 is obviously implausible.

Leave-one-out analyses are better framed as robustness checks and as a way to identify sources of study heterogeneity that you may have missed otherwise. You've set your inclusion/exclusion criteria for your meta-analysis for a reason, so if a study fulfills criteria, then it should be included.

Outlier analyses are also a good check to make sure that you have entered numbers correctly. A common reason for mistakes in meta-analysis is that standard error and standard deviation get mixed up. Of course, this should be carefully checked for all studies, but outlier analysis is a good backstop, just in case.

It's worthwhile reporting your leave-one-out analysis, regardless of whether you discover outliers. If your result hangs on the inclusion of a single study, then your findings aren't terribly robust and this should be made clear in the discussion of your results.

How to perform a Bayesian meta-analysis in R

Fuente: <https://www.dsquintana.blog/how-to-perform-a-bayesian-meta-analysis-in-r/>

Meta-analyses conventionally adopt a frequentist approach, in which a p-value is used to make a binary decision regarding the presence of an effect. While correctly used p-values can be useful, using a Bayesian framework can provide a number of benefits for the meta-analyst. However, Bayesian meta-analysis is uncommon, probably due to a historical lack of accessible software options for performing these tests. So in this post I'm going to outline the benefits of Bayesian meta-analysis and a step-by-step guide (with code) on to perform one using the relatively new "RoBMA" R package.

I've previously outlined the benefits of using a Bayesian framework for hypothesis testing in a 2018 paper, but there are two benefits that are particularly interesting for meta-analysis, which are worth covering before diving into the code.

First, Bayes factors are an intuitive metric for describing and understanding the relative evidence for two models, typically an alternative model vs. a null model. So using this approach, one could say that there's 12 times more evidence for an alternative model compared to a null model, for instance. Of course, frequentist meta-analysis can be reported with effect sizes, but interpreting effect sizes can be difficult, as Cohen's benchmarks of small/medium/large effects are not universal across research fields.

Second, non-significant p-values carry very little information, as these values can either suggest the absence of evidence or evidence of absence, regardless of the size of your p-value. As alluded to above, with Bayes factors you can also assess the relative evidence for a null model compared to an alternative model (unlike null hypothesis testing in which you can only reject a null hypothesis). Along with estimating the relative evidence for meta-analytic models, Bayes factors can also be used to estimate the relative evidence for publication bias. Frequentist approaches for publication suffer this same limitation, in that a non-significant test for publication bias can either suggest the absence of evidence or evidence of absence.

To illustrate the differences between a frequentist and Bayesian meta-analysis, I'm going to use an example that I've used for a previous tutorial paper for a frequentist meta-analysis. This dataset contains 17 studies that report the correlation between medication adherence and a measure of conscientiousness.

Let's run a frequentist analysis and construct a forest plot to visualise the result using the following script:

```
library(metafor) #load the metafor package

dat <- dat.molloy2014 # Extract the dataset

dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat.molloy2014,
```

```

slab=paste(authors, year, sep=", ")

res <- rma(yi, vi, data=dat) # Perform meta-analysis
res # print results

library(metafor) #load the metafor package

dat <- dat.molloy2014 # Extract the dataset

dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat.molloy2014,
              slab=paste(authors, year, sep=", "))

res <- rma(yi, vi, data=dat) # Perform meta-analysis
res # print results

```

Here are the results, followed by a forest plot:

```

Random-Effects Model (k = 16; tau^2 estimator: REML)

tau^2 (estimated amount of total heterogeneity): 0.0081 (SE = 0.0055)
tau (square root of estimated tau^2 value):      0.0901
I^2 (total heterogeneity / total variability):    61.73%
H^2 (total variability / sampling variability):    2.61

```

```

Test for Heterogeneity:
Q(df = 15) = 38.1595, p-val = 0.0009

```

Model Results:

estimate	se	zval	pval	ci.lb	ci.ub
0.1499	0.0316	4.7501	<.0001	0.0881	0.2118 ***

```

Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

Random-Effects Model (k = 16; tau^2 estimator: REML)

```

```

tau^2 (estimated amount of total heterogeneity): 0.0081 (SE = 0.0055)
tau (square root of estimated tau^2 value):      0.0901
I^2 (total heterogeneity / total variability):    61.73%
H^2 (total variability / sampling variability):    2.61

```

```

Test for Heterogeneity:
Q(df = 15) = 38.1595, p-val = 0.0009

```

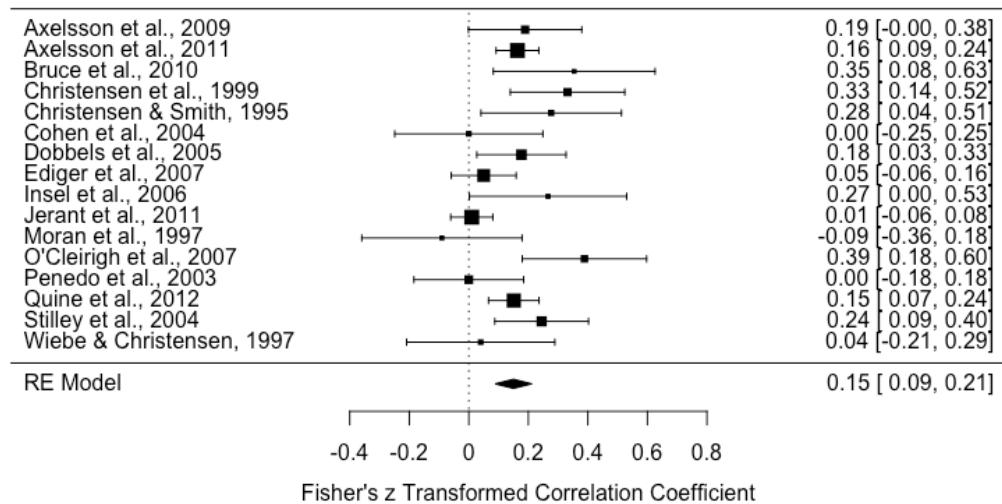
Model Results:

estimate	se	zval	pval	ci.lb	ci.ub
0.1499	0.0316	4.7501	<.0001	0.0881	0.2118 ***

```

Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```



Of note, there was significant heterogeneity [$Q(df = 15) = 38.1595$, $p\text{-value} = 0.0009$] and the effect size estimate was 0.15 ($p < .001$).

Conventionally, a meta-analyst would construct a funnel plot and perform Egger's regression test to assess publication bias via funnel plot asymmetry. But as I've outlined before, funnel plot asymmetry can only assess small study bias which can include publication bias, but also other factors (e.g., study quality).

```
funnel(res)

regtest(res)

# regtest results #

Regression Test for Funnel Plot Asymmetry

Model:      mixed-effects meta-regression model
Predictor: standard error

Test for Funnel Plot Asymmetry: z = 1.0216, p = 0.3070
Limit Estimate (as sei -> 0):  b = 0.0790 (CI: -0.0686, 0.2266)

funnel(res)

regtest(res)

# regtest results #

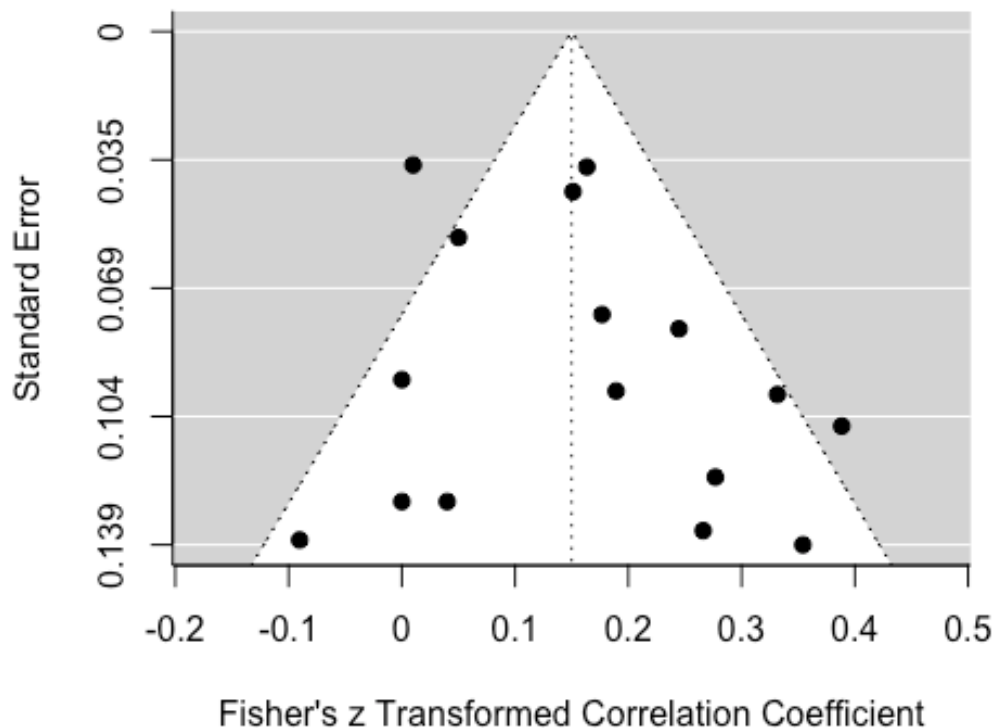
Regression Test for Funnel Plot Asymmetry

Model:      mixed-effects meta-regression model
```

Predictor: standard error

Test for Funnel Plot Asymmetry: $z = 1.0216$, $p = 0.3070$

Limit Estimate (as sei $\rightarrow 0$): $b = 0.0790$ (CI: $-0.0686, 0.2266$)



Eyeballing the funnel plot suggests that the distribution of effects is fairly symmetrical. Egger's regression test was non-significant, which suggest either that there is either no asymmetry or the data were not sensitive enough.

Next, let's use an approach that has been designed to specifically assess publication bias: weight-function models. In short, weight-function models provide a greater weight to studies that were less likely to have been published, based on their associated p value, for the estimation of a summary effect size. For example, it is a fair assumption that a study with a p value of 0.001 is more likely to be published than a study with a p value of 0.45. Other methods for publication bias detection and correction have been proposed, but weight-function models seem to perform the best across a range of situations (e.g., high heterogeneity).

It's worth emphasizing two limitations with selection models: 1) These models assume that the chosen selection model is an good approximation for the underlying selection process (i.e., that the chances of publication depend on the p value). But despite this limitation,

weight-function models at least make the selection process is explicit (whereas this the selection process is implicit in other methods, which means they can't be evaluated). 2) Selection-models (like other publication bias methods) are a type of sensitivity analysis, which means that the publication bias model is an estimate that isn't based on real numbers, but a set of assumptions for the how the chances of publication depends on the p value.

There are various approaches for selection models (e.g., a beta model, a power model), but here we will use the Vevea and Hedges (1995) step function approach, which is implemented in the weightr package:

```
weightr

library(weightr)

weightfunct(dat$yi, dat$vi)

Unadjusted Model (k = 16):

tau^2 (estimated amount of total heterogeneity): 0.0070 (SE = 0.0051)
tau (square root of estimated tau^2 value): 0.0834

Test for Heterogeneity:
Q(df = 15) = 38.1595, p-val = 0.001436053

Model Results:

      estimate std.error z-stat      p-val    ci.lb    ci.ub
Intercept  0.1486   0.03073  4.835 1.3288e-06 0.08837 0.2088

Adjusted Model (k = 16):

tau^2 (estimated amount of total heterogeneity): 0.0056 (SE = 0.0045)
tau (square root of estimated tau^2 value): 0.0750

Test for Heterogeneity:
Q(df = 15) = 38.1595, p-val = 0.001436053

Model Results:

      estimate std.error z-stat      p-val    ci.lb    ci.ub
Intercept  0.09153   0.04464  2.050 0.040341  0.00403 0.1790
0.025 < p < 1 0.24121   0.20122  1.199 0.230626 -0.15317 0.6356

Likelihood Ratio Test:
X^2(df = 1) = 2.98493, p-val = 0.084043

library(weightr)

weightfunct(dat$yi, dat$vi)

Unadjusted Model (k = 16):
```

```
tau^2 (estimated amount of total heterogeneity): 0.0070 (SE = 0.0051)
tau (square root of estimated tau^2 value): 0.0834
```

Test for Heterogeneity:

```
Q(df = 15) = 38.1595, p-val = 0.001436053
```

Model Results:

	estimate	std.error	z-stat	p-val	ci.lb	ci.ub
Intercept	0.1486	0.03073	4.835	1.3288e-06	0.08837	0.2088

Adjusted Model (k = 16):

```
tau^2 (estimated amount of total heterogeneity): 0.0056 (SE = 0.0045)
tau (square root of estimated tau^2 value): 0.0750
```

Test for Heterogeneity:

```
Q(df = 15) = 38.1595, p-val = 0.001436053
```

Model Results:

	estimate	std.error	z-stat	p-val	ci.lb	ci.ub
Intercept	0.09153	0.04464	2.050	0.040341	0.00403	0.1790
0.025 < p < 1	0.24121	0.20122	1.199	0.230626	-0.15317	0.6356

Likelihood Ratio Test:

```
X^2(df = 1) = 2.98493, p-val = 0.084043
```

This output might be new for many people reading, so I'm going to walk through it.

The "model results" section displays the conventional results, in terms of heterogeneity and effect size estimates. Note that the results are not exactly the same as our first analysis above, this is because the `weightr` package converts correlations to Cohen's d effect sizes for the analysis, and then transforms these results back to correlations.

`weightr`

The "adjusted model" section includes results accounting for publication bias (note the results in the "intercept" row). The summary effect estimate has shrunk from 0.15 to 0.09, and it is now very close to the threshold of statistical significance. ($p = 0.04$). A formal comparison of these two models, which tests for publication bias, was statistically significant using a threshold of $p = 0.1$, which has been recommended by Renkewitz & Keiner, along with Begg & Mazumdar.

This shifted threshold highlights the arbitrary nature of p-value thresholds. Strictly using the conventional significance threshold of $p = 0.05$, one would conclude that there was no publication bias. However, using a significance threshold of $p = 0.1$, one conclude that there's indeed publication bias.

As mentioned above, but worth repeating, Bayes factors are values that represent the relative evidence for two competing models, without requiring a strict cutoff. This means

that we can do away with the need for making a binary decision when it comes to evaluating the presence of publication bias.

So let's now jump in and perform Bayesian hypothesis tests using the RoBMA R package. This package generates three families of models, across 12 models, then averages their performance. These models vary on whether they're fixed or random effects models (i.e., assuming heterogeneity), a null or alternative hypothesis models, or whether models include the presence of publication bias or not. In terms of publication bias models, separate models also use a two-step approach (significant or non-significant) or a three-step approach (significant, marginally significant, or non-significant). For more information, see Maier and colleagues.

RoBMA

Before running this analysis, just keep in mind that this can take some time, depending on the speed of your computer. It took my five-year-old MacBook Pro just over five minutes to complete these analyses, for reference.

```
fit <- RoBMA(r = dat$ri,
            n = dat$ni,
            study_names = dat$authors,
            seed = 2333)

summary(fit)

## OUTPUT ###

Call:
RoBMA(r = dat$ri, n = dat$ni, study_names = dat$authors, seed = 2333)

Robust Bayesian Meta-Analysis

      Models Prior prob. Post. prob. Incl. BF
Effect      6/12      0.500      0.992  119.310
Heterogeneity 6/12      0.500      0.963   25.726
Pub. bias     8/12      0.500      0.544    1.194

Model-averaged estimates
      Mean Median 0.025 0.975
mu      0.129  0.129 0.057 0.202
tau      0.163  0.158 0.000 0.320
omega[0,0.05] 1.000  1.000 1.000 1.000
omega[0.05,0.1] 0.814  0.956 0.252 1.000
omega[0.1,1] 0.718  0.848 0.153 1.000
(Tau is on Cohen's d scale.)
(Estimated omegas correspond to two-sided p-values)

fit <- RoBMA(r = dat$ri,
            n = dat$ni,
            study_names = dat$authors,
            seed = 2333)

summary(fit)
```

```
## OUTPUT ###
```

```
Call:
```

```
RoBMA(r = dat$ri, n = dat$ni, study_names = dat$authors, seed = 2333)
```

```
Robust Bayesian Meta-Analysis
```

	Models	Prior prob.	Post. prob.	Incl. BF
Effect	6/12	0.500	0.992	119.310
Heterogeneity	6/12	0.500	0.963	25.726
Pub. bias	8/12	0.500	0.544	1.194

```
Model-averaged estimates
```

	Mean	Median	0.025	0.975
mu	0.129	0.129	0.057	0.202
tau	0.163	0.158	0.000	0.320
omega[0,0.05]	1.000	1.000	1.000	1.000
omega[0.05,0.1]	0.814	0.956	0.252	1.000
omega[0.1,1]	0.718	0.848	0.153	1.000

(Tau is on Cohen's d scale.)
(Estimated omegas correspond to two-sided p-values)

Let's go through these results, first addressing the "Robust Bayesian meta-analysis" section. The "Effect" row includes a Bayes Factor ("Incl. BF") of 119, which indicates that there is 119 times more evidence for an effect model relative to a null model. The "Heterogeneity" row indicates that there is 26 times more evidence for a heterogeneity model than a model without heterogeneity. Finally, the "Pub. bias" row indicates that there's 1.2 times more evidence for a publication model than a model without publication bias, which is very small evidence.

The "Model-averaged estimates" section contains an estimate of the summary effect size (μ), which was 0.13. Keep in mind that this is the values that is averaged across all models, including publication bias models, so this is a publication bias adjusted value.

Let's compare this model-averaged estimate against the a model that parallels the conventional frequentist approach: A model that assumes an effect with heterogeneity and no publication bias (i.e., a random-effects meta-analysis). This is model #10 in our analysis, and information for this model can be found by entering this command: `summary(fit, type = "individual")`. This command will list information from all 12 models (output not shown here for the sake of space). For model #10, the mean estimate was 0.15, which mirrors our conventional frequentist analysis above.

```
summary(fit, type = "individual")
```

The "Model-averaged estimates" section also provides interesting information regarding the relative probability of publication of results of significant (the `omega[0,0.05]` row), marginally significant (the `omega[0.05,0.1]` row), and non-significant (the `omega[0.1,1]` row) results. This analysis suggested there was 100% probability that significant results ($p < 0.05$) were published, which is a consequence of how the model was set up, a 81.4% probability that marginally significant results ($.05 < p < .10$) were published (relative to

significant studies), and 71.8% chance that non-significant results ($p > 0.1$) were published (relative to significant studies).

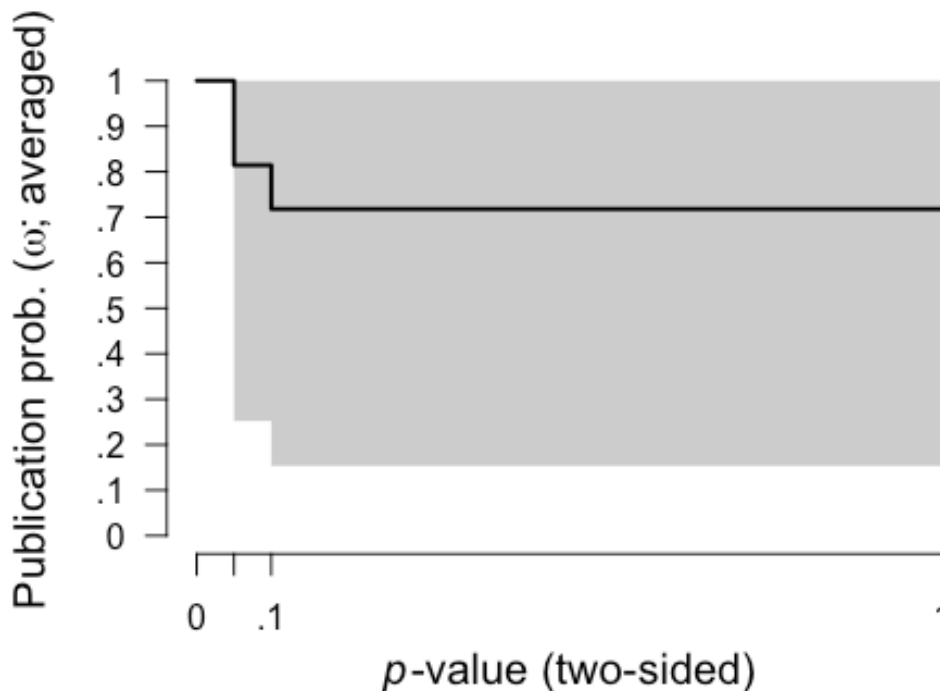
```
omega[0,0.05]
```

```
omega[0.05,0.1]
```

```
omega[0.1,1]
```

We can also plot these results using this command: `plot(fit, parameter = "omega")`

```
plot(fit, parameter = "omega")
```



It's also important to check the diagnostics of these 12 models to identify any issues. Rhat convergence and the effect sampling size (ESS) provide a summary of Markov chain Monte Carlo (MCMC) convergence for the appropriateness of drawing inferences. It's recommended that Rhat convergence values should be under 1.05 and that the effect sampling size (ESS) is sufficiently large. There's no clear rule-of-thumb for what constitutes a "large" ESS, but even an ESS of a few hundred is fine if you're only interested in the posterior means. We can easily generate a summary including diagnostics with this code:

```
summary(fit, type = "models", diagnostics = TRUE)
```

```
# OUTPUT WITH SOME INFO OMITTED FOR SPACE #
```

```
Models diagnostics overview
```

	Prior mu	Prior tau
1	Spike(0)	Spike(0)
2	Spike(0)	Spike(0)

```

3          Spike(0)          Spike(0)
4          Spike(0) InvGamma(1, 0.15)[0, Inf]
5          Spike(0) InvGamma(1, 0.15)[0, Inf]
6          Spike(0) InvGamma(1, 0.15)[0, Inf]
7 Normal(0, 1)[-Inf, Inf]          Spike(0)
8 Normal(0, 1)[-Inf, Inf]          Spike(0)
9 Normal(0, 1)[-Inf, Inf]          Spike(0)
10 Normal(0, 1)[-Inf, Inf] InvGamma(1, 0.15)[0, Inf]
11 Normal(0, 1)[-Inf, Inf] InvGamma(1, 0.15)[0, Inf]
12 Normal(0, 1)[-Inf, Inf] InvGamma(1, 0.15)[0, Inf]
          Prior omega max(MCMC error) min(ESS) max(Rhat)
1          Spike(1)          NA          NA          NA
2          Two-sided((0.05), (1, 1))          0.000          7776          1.000
3 Two-sided((0.1, 0.05), (1, 1, 1))          0.002          8239          1.001
4          Spike(1)          0.001          18675          1.000
5          Two-sided((0.05), (1, 1))          0.003          7924          1.001
6 Two-sided((0.1, 0.05), (1, 1, 1))          0.002          7786          1.000
7          Spike(1)          0.000          30050          1.000
8          Two-sided((0.05), (1, 1))          0.002          9407          1.001
9 Two-sided((0.1, 0.05), (1, 1, 1))          0.002          8700          1.000
10          Spike(1)          0.001          12525          1.000
11          Two-sided((0.05), (1, 1))          0.002          9540          1.000
12 Two-sided((0.1, 0.05), (1, 1, 1))          0.002          8049          1.001

```

```
summary(fit, type = "models", diagnostics = TRUE)
```

```
# OUTPUT WITH SOME INFO OMITTED FOR SPACE #
```

```
Models diagnostics overview
```

```

          Prior mu          Prior tau
1          Spike(0)          Spike(0)
2          Spike(0)          Spike(0)
3          Spike(0)          Spike(0)
4          Spike(0) InvGamma(1, 0.15)[0, Inf]
5          Spike(0) InvGamma(1, 0.15)[0, Inf]
6          Spike(0) InvGamma(1, 0.15)[0, Inf]
7 Normal(0, 1)[-Inf, Inf]          Spike(0)
8 Normal(0, 1)[-Inf, Inf]          Spike(0)
9 Normal(0, 1)[-Inf, Inf]          Spike(0)
10 Normal(0, 1)[-Inf, Inf] InvGamma(1, 0.15)[0, Inf]
11 Normal(0, 1)[-Inf, Inf] InvGamma(1, 0.15)[0, Inf]
12 Normal(0, 1)[-Inf, Inf] InvGamma(1, 0.15)[0, Inf]
          Prior omega max(MCMC error) min(ESS) max(Rhat)
1          Spike(1)          NA          NA          NA
2          Two-sided((0.05), (1, 1))          0.000          7776          1.000
3 Two-sided((0.1, 0.05), (1, 1, 1))          0.002          8239          1.001
4          Spike(1)          0.001          18675          1.000
5          Two-sided((0.05), (1, 1))          0.003          7924          1.001
6 Two-sided((0.1, 0.05), (1, 1, 1))          0.002          7786          1.000
7          Spike(1)          0.000          30050          1.000
8          Two-sided((0.05), (1, 1))          0.002          9407          1.001
9 Two-sided((0.1, 0.05), (1, 1, 1))          0.002          8700          1.000
10          Spike(1)          0.001          12525          1.000
11          Two-sided((0.05), (1, 1))          0.002          9540          1.000
12 Two-sided((0.1, 0.05), (1, 1, 1))          0.002          8049          1.001

```

For all of our models, the largest Rhat convergence values [the max(Rhat) column] column was 1.001 and the smallest ESS [min(ESS) column] was 7776, so we can be confident of good MCMC convergence, and be on our way. If Rhat convergence values are over 1.05 or ESS values are below a few hundred, then you can try re-running your analysis with an increased number of sampling iterations and burnin. Of course, this will take longer to compute.

Keep in mind that these analyses are using a series of default prior distributions, which are designed to represent typical effects in the psychological sciences. For details on adjusting the your prior distributions, see Maier and colleagues.

In summary, our analysis revealed only very small evidence in favour of presence publication bias. Consequently, there is was only a very modest reduction in the size of the summary effect estimate after adjusting for publication bias.

So what have we learned here? First, common approaches for assessing publication bias don't actually assess publication bias. A weighted-model approach can be used as a sensitivity measure to assess the potential presence of publication bias in meta-analysis, but this using this approach via a frequentist framework has some limitations (e.g., a non-significant result is not terribly informative). Second, a model-averaged Bayesian meta-analysis can assess the relative evidence for publication bias and provide a publication bias-adjusted estimate. This meta-analysis is relatively straightforward to perform via the RoBMA R package.

[RoBMA](#)

Introducing the {metameta} R package

Fuente: <https://www.dsquintana.blog/metameta-r-package-meta-analysis/>

I'm happy to announce that I just released my first R package. Here's a quick rundown on how you can use the {metameta} package to effortlessly calculate the statistical power of published meta-analyses to better understand the evidential value of included studies.

First we'll install the package via Github and then load it. The package contains two main features:

1. Functions to calculate the statistical power of studies in a meta-analysis
2. A function to create a Firepower plot, which visualises statistical power across meta-analyses

```
devtools::install_github("dsquintana/metameta")  
library(metameta)
```

```
devtools::install_github("dsquintana/metameta")  
library(metameta)
```

For first example, we're going to extract some data from this forest plot published in this meta-analysis on the impact of intranasal oxytocin on social cognition. All we need is the effect size and confidence interval data for each study included in the meta-analysis.

At a minimum, the dataset for analysis needs three columns, with the following labels:

1. "yi" for the effect size
2. "lower" for the lower CI bound
3. "upper" for the upper CI bound

You can also add a column for the study name, but this isn't strictly necessary.

This is what your datafile should look like:

Assuming we've named this dataset "dat_keeck", we're going to use this in the mapower_ul function. This requires three arguments:

```
mapower_ul
```

1. The data (structured as shown above)
2. The observed summary effect size estimate
3. The name of the meta-analysis (required for the other core function we'll get to soon)


```

power_keech <- mapower_ul(dat = dat_keech,
                          observed_es = 0.08,
                          name = "Keech et al 2017")
power_keech_dat <- power_keech$dat
power_keech_dat

power_keech <- mapower_ul(dat = dat_keech,
                          observed_es = 0.08,
                          name = "Keech et al 2017")
power_keech_dat <- power_keech$dat
power_keech_dat

```

This will give us statistical power for a range of possible "true" effect sizes: The reported summary effect size ("power_es_observed") and a range of effect sizes from 0.1 to 1 (in increments on 0.1).

In other words, this analysis calculates how likely it would be to observe a significant effect given a range of possible effect sizes.

In this particular field, the average effect size is around 0.2, and that's probably inflated due to publication bias. So conservatively assuming that 0.2 is the true effect size, power ranges from 7% to 30% in these studies.

There's also a function for meta-analyses that report effect sizes and standard errors, called `mapower_se`. To illustrate, let's extract the data from this forest plot published in this meta-analysis.

```
mapower_se
```

For this function, only two columns are required:

1. "yi" for effect sizes
2. "sei" for standard errors

Assuming we've named this dataset "dat_ooi", we're going to use this in the `mapower_se` function. This requires three arguments, as before:

```
mapower_se
```

- The data
- The observed summary effect size estimate
- The name of the meta-analysis

```

power_ooi <- mapower_se(dat = dat_ooi,
                        observed_es = 0.178,
                        name = "ooi et al 2017")
power_ooi_dat <- power_ooi$dat
power_ooi_dat

power_ooi <- mapower_se(dat = dat_ooi,
                        observed_es = 0.178,
                        name = "ooi et al 2017")
power_ooi_dat <- power_ooi$dat
power_ooi_dat

```

As before, this will return the statistical power for a range of possible "true" effect sizes.

Sometimes it's useful to calculate power for a body of meta-analyses, which might be reported in the same article or across articles. But illustrating the power of individual studies from multiple meta-analyses can be difficult to interpret if there are lots of studies.

An alternative is to illustrate the power per meta-analysis by calculating the median power across studies. We can illustrate this with a "Firepower" plot, which we can create using the `firepower` function. First, we need to prepare the data.

`firepower`

Here, we're combining three meta-analyses into a list.

```

### Calculate median power for three meta-analyses
power_ooi <- mapower_se(dat = dat_ooi,
                        observed_es = 0.178,
                        name = "ooi et al 2017")
power_med_ooi <- power_ooi$power_median_dat

keech_power <- mapower_ul(dat = dat_keech,
                          observed_es = 0.08,
                          name = "Keech et al 2017")
power_med_keech <- keech_power$power_median_dat

power_bakermans_kranenburg <-
  mapower_se(dat = dat_bakermans_kranenburg,
             observed_es = 0.32,
             name = "Bakermans-Kranenburg et al 2013")
power_med_bakermans_kranenburg <- power_bakermans_kranenburg$power_median_dat

### Create a list
list_power <- list(power_med_ooi, power_med_keech,
                  power_med_bakermans_kranenburg)

### Run firepower function
fp <- firepower(list_power)

### Create firepower plot

```

```

fp_plot <- fp$fp_plot
fp_plot

### Calculate median power for three meta-analyses
power_ooi <- mapower_se(dat = dat_ooi,
                        observed_es = 0.178,
                        name = "ooi et al 2017")
power_med_ooi <- power_ooi$power_median_dat

keech_power <- mapower_ul(dat = dat_keech,
                          observed_es = 0.08,
                          name = "Keech et al 2017")
power_med_keech <- keech_power$power_median_dat

power_bakermans_kranenburg <-
  mapower_se(dat = dat_bakermans_kranenburg,
             observed_es = 0.32,
             name = "Bakermans-Kranenburg et al 2013")
power_med_bakermans_kranenburg <- power_bakermans_kranenburg$power_median_dat

### Create a list
list_power <- list(power_med_ooi, power_med_keech,
                   power_med_bakermans_kranenburg)

### Run firepower function
fp <- firepower(list_power)

### Create firepower plot
fp_plot <- fp$fp_plot
fp_plot

```

And here's our Firepower plot

You can also extract the raw data underlying this plot:

```

library(tibble)
fpdata <- fp_plot$data
fpdata <- as_tibble(fpdata)
fpdata

library(tibble)
fpdata <- fp_plot$data
fpdata <- as_tibble(fpdata)
fpdata

```

```
# A tibble: 33 x 4
  meta_analysis_name      effect      power es_cat
  <chr>                <fct>    <dbl> <chr>
1 ooi et al 2017      es_observed 0.0808 observed_es
2 Keech et al 2017    es_observed 0.0602 observed_es
3 Bakermans-Kranenburg et al 2013 es_observed 0.187 observed_es
4 ooi et al 2017      es01       0.0596 range_es
5 Keech et al 2017    es01       0.0660 range_es
6 Bakermans-Kranenburg et al 2013 es01       0.0628 range_es
7 ooi et al 2017      es02       0.0891 range_es
8 Keech et al 2017    es02       0.115 range_es
9 Bakermans-Kranenburg et al 2013 es02       0.102 range_es
10 ooi et al 2017     es03       0.140 range_es
# ... with 23 more rows
```

Try out this package for yourself either using the three example datasets or your own data. If you find any bugs or have feature requests, please file an issue on the package's GitHub page.

Equivalence-enhanced forest plots with power tiles for meta-analysis

Fuente: <https://www.dsquintana.blog/equivalence-enhanced-forest-plots-with-power-tiles/>

Unlike primary research, most meta-analysis articles provide enough information to recreate their results. This information tends to be reported in forest plots, which provide a summary of the extracted effect size from each study, along with some measure of variance, like standard error or a confidence interval.

As well as facilitating the verification of the results, this easily extractable information also provides the opportunity for the reanalysis of data.

Some meta-analyses report a non-significant summary effect size and conclude that there is no effect. However, a non-significant result only means that you failed to reject the null hypothesis, not that there was support for the null hypothesis. However, equivalence tests make it possible to reject of effects as large, or larger, than a smallest effect size of interest. For instance, if you believe that an effect of $d = 0.2$ or smaller is practically meaningless, then you can perform an equivalence test to reject effects that are as large, or larger, than $d = 0.2$. If the result of this test is statistically significant, then you could conclude that the effect was statistically equivalent.

Equivalence tests are becoming popular in primary research studies, but I've yet to see one "out in the wild" for a meta-analysis. Fortunately, Daniel Lakens' TOSTER R package provides a function for performing equivalence tests for meta-analytic results.

Let's run a test on a meta-analysis with a summary effect size of 0.07 and a standard error of 0.0918. We'll set our equivalence bounds at 0.2.

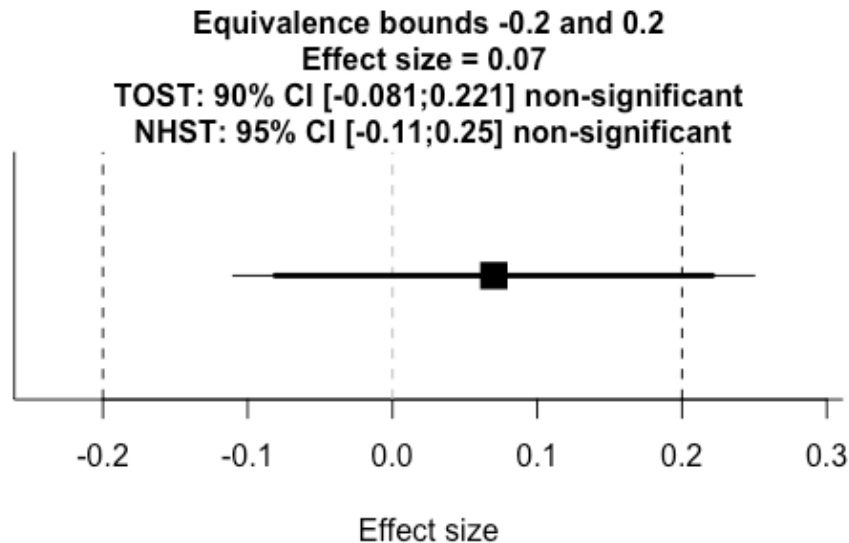
```
library(TOSTER)

TOSTmeta(ES = 0.07,
se = 0.0918,
low_eqbound_d = -0.2,
high_eqbound_d = 0.2)

library(TOSTER)

TOSTmeta(ES = 0.07,
se = 0.0918,
low_eqbound_d = -0.2,
high_eqbound_d = 0.2)
```

Running this test yields a figure like this.



We can see at a glance that the null hypothesis significance test (NHST) was not significant because the 95% confidence interval (thin line) crossed zero. Additionally, we can see the two one sided test (TOST) for equivalence (thick line) was not significant as one of the 90% confidence bounds crossed our equivalence threshold. This is a useful plot, but there's no straightforward way to visualise a series of tests with various equivalence bounds.

There are two reasons you might want to combine a series of tests. First, you might have several meta-analysis results from a single paper you would like to combine into a single figure. Second, you might want to combine results from different papers that address similar research questions.

I'm working on a project that's addressing the second question. I couldn't find a good solution for visualising equivalence tests for a series of summary effect sizes, so I made my own R function, which you can access on my Github repository.

So now we're going to run a function to analyse the data (also available on my Github repository).

```
leppanen_2017_1_results <- ma_pipe_sei(
  "dat_leppanen_2017_1.csv", # Load data
  true_effect = 0.09, # Reported effect size
  rep_lower = -0.06, # Reported lower bound
  rep_upper = 0.24, # Reported upper bound
  analysis_title = "Leppanen_2017_1", # Title of analysis
  plot = TRUE
)

leppanen_2017_1_et_dat <- leppanen_2017_1_results$et # Extract equivalence test results

leppanen_2017_1_results <- ma_pipe_sei(
  "dat_leppanen_2017_1.csv", # Load data
```

```

    true_effect = 0.09, # Reported effect size
    rep_lower = -0.06, # Reported lower bound
    rep_upper = 0.24, # Reported upper bound
    analysis_title = "Leppanen_2017_1", # Title of analysis
    plot = TRUE
  )

leppanen_2017_1_et_dat <- leppanen_2017_1_results$et # Extract equivalence test
results

```

The function `ma_pipe_sei` takes the extracted data of a meta-analysis, and has to include two columns: "yi" with effect size data, "lower" with the lower confidence interval bound, and "upper" with the upper confidence interval bound.

```
ma_pipe_sei
```

This function calculates the standard error for each study effect based on the upper and lower confidence interval bounds. It performs meta-analysis, outputs the data, creates a sunset plot, performs an equivalence test, and visualises the equivalence test. We can extract these outputs one-by-one into new objects.

```

leppanen_2017_1_res <- leppanen_2017_1_results$res # Meta-analysis results

leppanen_2017_1_data <- leppanen_2017_1_results$dat # Meta-analysis raw data

leppanen_2017_1_sunset <- leppanen_2017_1_results$sunset # Meta-analysis sunset
plot

leppanen_2017_1_et_plot <- leppanen_2017_1_results$et_plot # Extract
equivalence test results

leppanen_2017_1_et_dat <- leppanen_2017_1_results$et # Extract equivalence test
results

leppanen_2017_1_res <- leppanen_2017_1_results$res # Meta-analysis results

leppanen_2017_1_data <- leppanen_2017_1_results$dat # Meta-analysis raw data

leppanen_2017_1_sunset <- leppanen_2017_1_results$sunset # Meta-analysis sunset
plot

leppanen_2017_1_et_plot <- leppanen_2017_1_results$et_plot # Extract
equivalence test results

leppanen_2017_1_et_dat <- leppanen_2017_1_results$et # Extract equivalence test
results

```

Now we just repeat this for other meta-analysis datasets (see my Github repository for additional example data and analysis scripts).

Now we can combine the results and create our plot.

```

com1 <- rbind(leppanen_2017_1_et_dat,
              leppanen_2017_2_et_dat,
              leppanen_2017_3_et_dat,
              leppanen_2017_4_et_dat,
              leppanen_2017_5_et_dat,
              leppanen_2017_6_et_dat)

com1 <- combine_et(com1)

com1 <- com1 + ggtitle("Summary effect sizes and equivalence bounds") +
  theme(plot.title = element_text(hjust = 0.5))

com1 <- rbind(leppanen_2017_1_et_dat,
              leppanen_2017_2_et_dat,
              leppanen_2017_3_et_dat,
              leppanen_2017_4_et_dat,
              leppanen_2017_5_et_dat,
              leppanen_2017_6_et_dat)

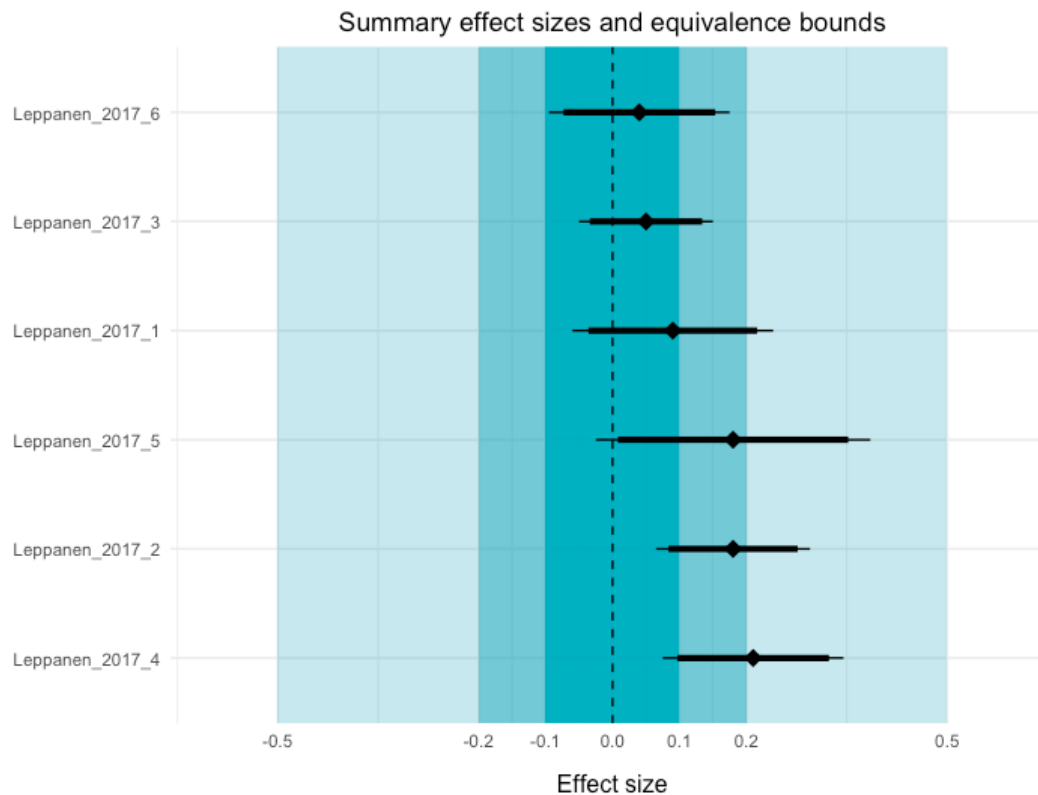
com1 <- combine_et(com1)

com1 <- com1 + ggtitle("Summary effect sizes and equivalence bounds") +
  theme(plot.title = element_text(hjust = 0.5))

```

This uses the `combine_et()` function, that's also in my script. Here, you just need to input a dataset that contains all the equivalence test results you would like to include. I've also added a title to the figure.

```
combine_et()
```

In this plot, I've visualised three equivalence test bounds: 0.1, 0.2, 0.5. You can also see the 95% NHST confidence intervals (thin lines) and the 90% TOST confidence intervals for each meta-analysis. If you would consider effects smaller than 0.2 to be practically unimportant, the first two meta-analyses demonstrate equivalent effects.

The second complementary visualisation that I've created, which I've called power tiles, illustrates the median statistical power of each study that was included in the meta-analysis. A meta-analysis that is comprised of studies with low statistical power is less credible than one that includes studies with high power. To create this function, I've adapted code from the 'metaviz' R package.

One element of this power analysis is that you need enter the "true" effect size for this calculation. As this is difficult in practice, I've included three options: The summary effect size calculated from the meta-analysis, an effect size of 0.33, and an effect size of 0.66. Here's the script you need to run, which will also combine the power tile plot with our equivalence-enhanced forest plot.

```
power_med <- rbind(leppanen_2017_1_power_median_dat,
  leppanen_2017_2_power_median_dat,
  leppanen_2017_3_power_median_dat,
  leppanen_2017_4_power_median_dat,
  leppanen_2017_5_power_median_dat,
  leppanen_2017_6_power_median_dat)

power_med_long <- gather(power_med, effect, power, observed:e66,
```

```

factor_key=TRUE)

tile <- ggplot(data = power_med_long, aes(x = effect, y = reorder(analysis, -
ES),)) +
  geom_tile(aes(fill = power)) +
  coord_equal(ratio = 0.8) +
  scale_fill_gradient(name = "Power", low = "#FDF0FF", high = "#E200FD") +
  theme_tufte(base_family="Helvetica")

tile <- tile + theme(axis.text.y = element_blank(),
                    axis.ticks.y = element_blank(),
                    axis.title.y = element_blank())

tile <- tile + ggtitle("Median statistical power") +
  xlab("Effect size") +
  theme(plot.title = element_text(hjust = 0.5))

tile <- tile + scale_x_discrete(labels=c("observed" = "Observed \n effect",
"e33" = "0.33",
                                     "e66" = "0.66"))

tile

# Combine the two plots
com1 + tile

power_med <- rbind(leppanen_2017_1_power_median_dat,
                  leppanen_2017_2_power_median_dat,
                  leppanen_2017_3_power_median_dat,
                  leppanen_2017_4_power_median_dat,
                  leppanen_2017_5_power_median_dat,
                  leppanen_2017_6_power_median_dat)

power_med_long <- gather(power_med, effect, power, observed:e66,
factor_key=TRUE)

tile <- ggplot(data = power_med_long, aes(x = effect, y = reorder(analysis, -
ES),)) +
  geom_tile(aes(fill = power)) +
  coord_equal(ratio = 0.8) +
  scale_fill_gradient(name = "Power", low = "#FDF0FF", high = "#E200FD") +
  theme_tufte(base_family="Helvetica")

tile <- tile + theme(axis.text.y = element_blank(),
                    axis.ticks.y = element_blank(),
                    axis.title.y = element_blank())

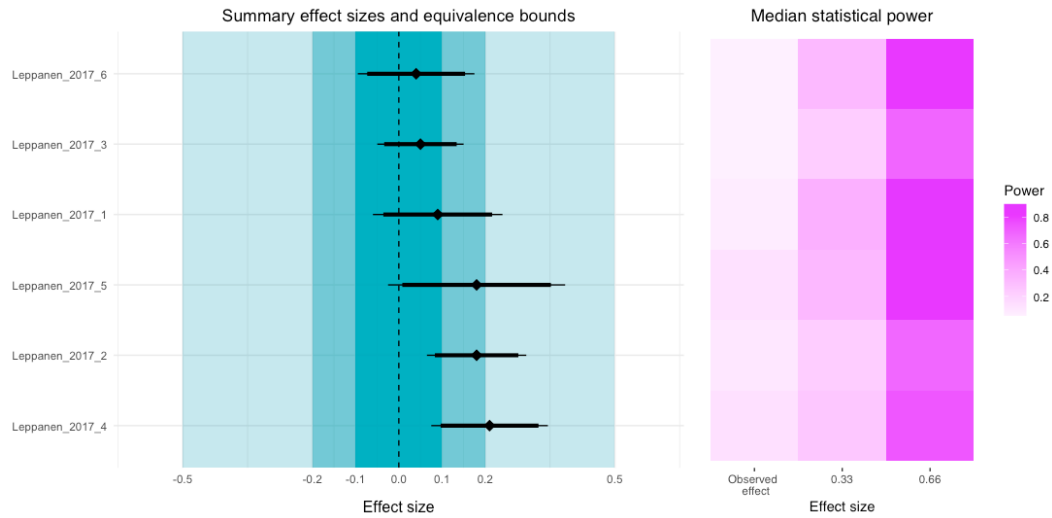
tile <- tile + ggtitle("Median statistical power") +
  xlab("Effect size") +
  theme(plot.title = element_text(hjust = 0.5))

tile <- tile + scale_x_discrete(labels=c("observed" = "Observed \n effect",
"e33" = "0.33",
                                     "e66" = "0.66"))

tile

```

```
# Combine the two plots
com1 + tile
```



Looking at the power tile plot, we can see that for the observed effects the median power was very low. However, if the true effect size was higher, then the power for each study (or to be more precise, the tests reported in the study) would be sufficient.

The code is a little messy, but I wanted to share an early version to get some feedback. Please let me know if you have any suggestions on how I can improve these visualisations.

Sunset funnel plots for meta-analysis

Fuente: <https://www.dsquintana.blog/meta-analysis-power-plot/>

Garbage in, garbage out is a common criticism of meta-analysis. While I'm generally a fan of meta-analysis, I think this is a very fair critique.

One approach to determine the evidential value of studies included in a meta-analysis is to determine the statistical power of the reported tests that were used. Tests with low statistical power have a low probability of finding a statistically significant effect, if there's a true effect to be found. This also means that tests with low statistical power are less likely to replicate. Consequently, a meta-analysis that contains studies with low power is likely to comprise results that won't replicate and a summary effect size estimate that is inflated.

You can calculate statistical power one-by-one for each effect size, but this is time consuming. However, it's possible to calculate statistical power using the information that has already been extracted for a meta-analysis, namely, a set of effect sizes and their variances.

To demonstrate this, I'm going to use the `sunset_viz` function from the `metaviz` R package. This function creates a sunset funnel plot, which is funnel plot variant that visualises the statistical power of each test included in the meta-analysis.

```
sunset_viz
```

```
metaviz
```

This function is relatively straightforward, however, you will need to specify the "true" population effect size. There are a few approaches you can take to specify this.

- Use the summary effect size from the meta-analysis

This is the most straightforward approach, and the default in this function, but this effect size is likely to be inflated due to publication bias.

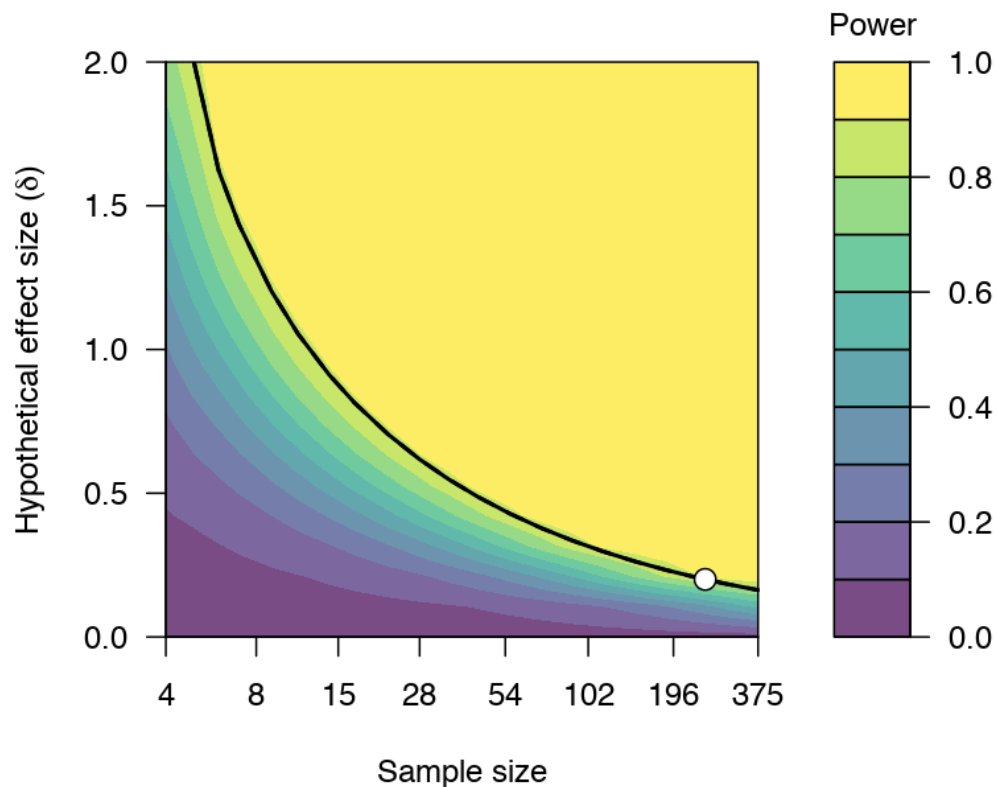
2. Use a publication-bias adjusted effect size

There are various tools, like `p-uniform`, that can be used to calculate a publication bias-adjusted effect size. However, these tools have some limitations. Namely, this should only be used for meta-analyses with low heterogeneity, and it can't be used for multilevel meta-analysis (e.g., meta-analyses that account for multiple effect sizes from individual studies).

3. Determine the smallest effect size of interest

The smallest effect size of interest (SESOI) is the smallest effect size you care about for the research question at hand. For example, if for your research question an effect size lower than $d = 0.1$ is essentially meaningless or has no practical value, then $d = 0.1$ is your SESOI. If your test is sensitive enough to detect the smallest effect size of interest, then it's able to detect larger effects as well.

For a quick demonstration, consider this plot below, which was created by the 'jpower' module in JAMOV. Let's say our SESOI is 0.1 and we have 250 participants for a paired samples t-test. This design can detect effect sizes of 0.2 with a probability of at least 0.88 (i.e., 88% power) when $\alpha=0.05$ (two-tailed). If the "true" effect size is higher than 0.2, then the probability of detecting these effect sizes are higher than 88%.



Now, to demonstrate this tool, let's use an example correlational dataset that I've used in previous demonstrations to perform a conventional meta-analysis.

```
library(metafor) #Load metafor
library(metaviz) #Load metaviz
library(punifrom) #Load puniform

dat <- get(data(dat.molloy2014)) # Extract data

dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat) #Calculate effect sizes

res <- rma.uni(yi, vi, data=dat)

forest_plot <- forest(res)
forest_plot
```

```

library(metafor) #Load metafor
library(metaviz) #Load metaviz
library(puniform) #Load puniform

dat <- get(data(dat.molloy2014)) # Extract data

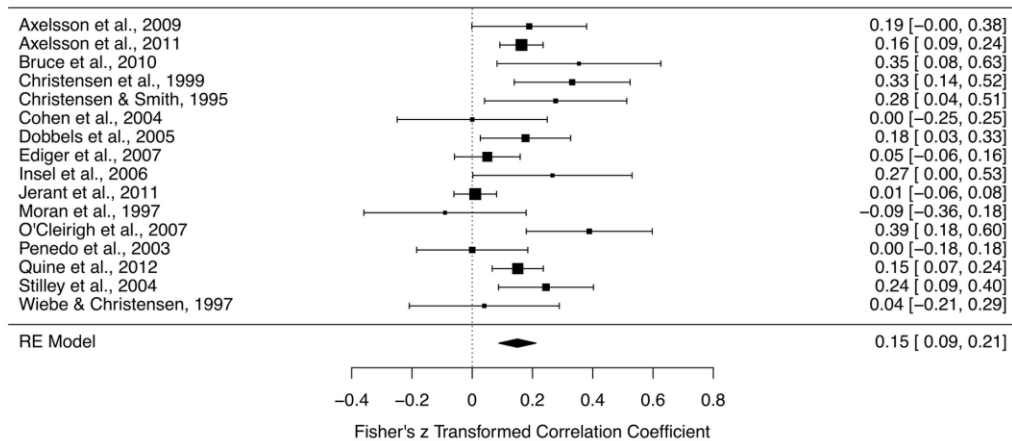
dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat) #Calculate effect sizes

res <- rma.uni(yi, vi, data=dat)

forest_plot <- forest(res)
forest_plot

```

Here are the results of this meta-analysis.



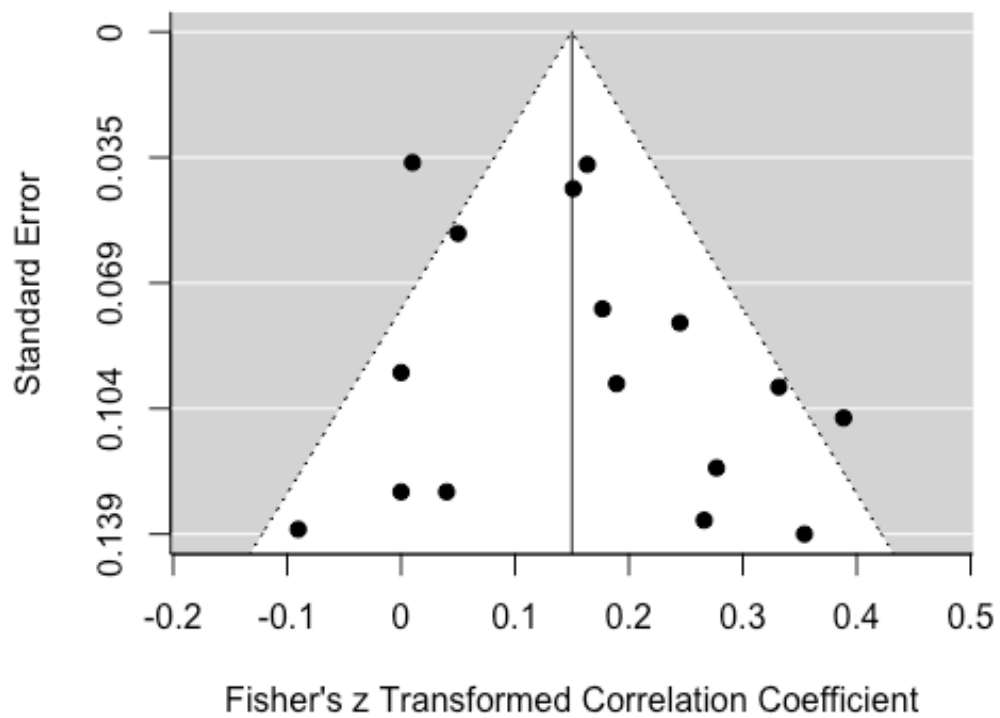
Let's also make a funnel plot, which illustrates the potential for small-study bias by plotting each effect size against its standard error, centred on the estimated summary effect size.

```

funnel_plot <- funnel(res)
funnel_plot

funnel_plot <- funnel(res)
funnel_plot

```



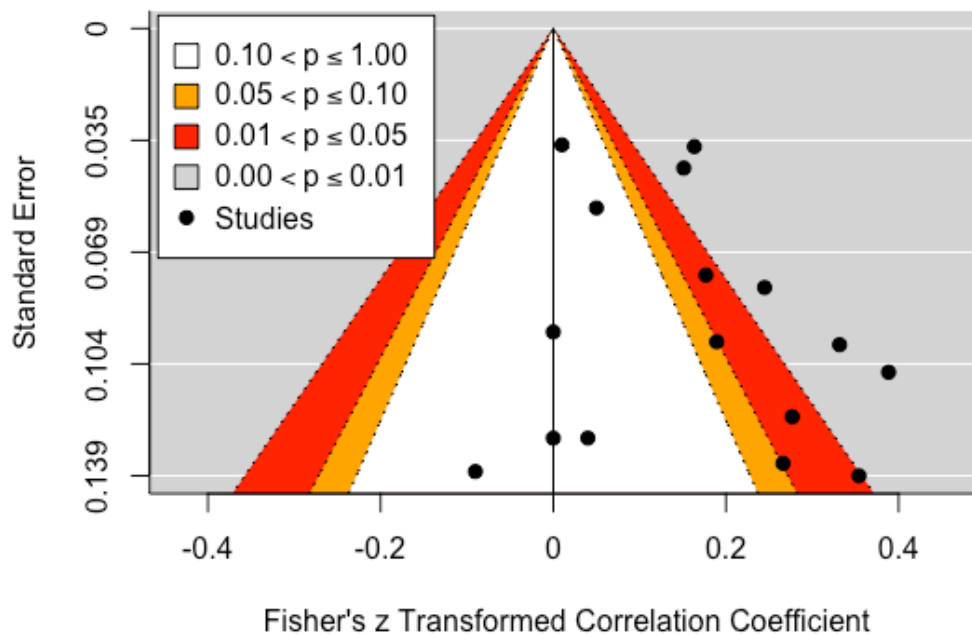
We can improve this plot by adding 'significance' contours and centring the funnel on zero.

```
funnel_contour <- funnel(res, level=c(90, 95, 99),
  shade=c("white", "orange", "red"),
  refline=0, legend="topleft")

funnel_contour

funnel_contour <- funnel(res, level=c(90, 95, 99),
  shade=c("white", "orange", "red"),
  refline=0, legend="topleft")

funnel_contour
```



This plot shows us that a few studies are lying in the 'significance' contours. While eyeballing this plot doesn't elicit too much concern for publication bias, this would still be a subjective conclusion.

To get a more objective understanding of how much publication bias is influencing the summary effect size estimate, we can adjust for publication bias using the `punif` package.

```
punif

puni_star(yi = dat$yi, vi = dat$vi,
          side = "right")

puni_star(yi = dat$yi, vi = dat$vi,
          side = "right")
```

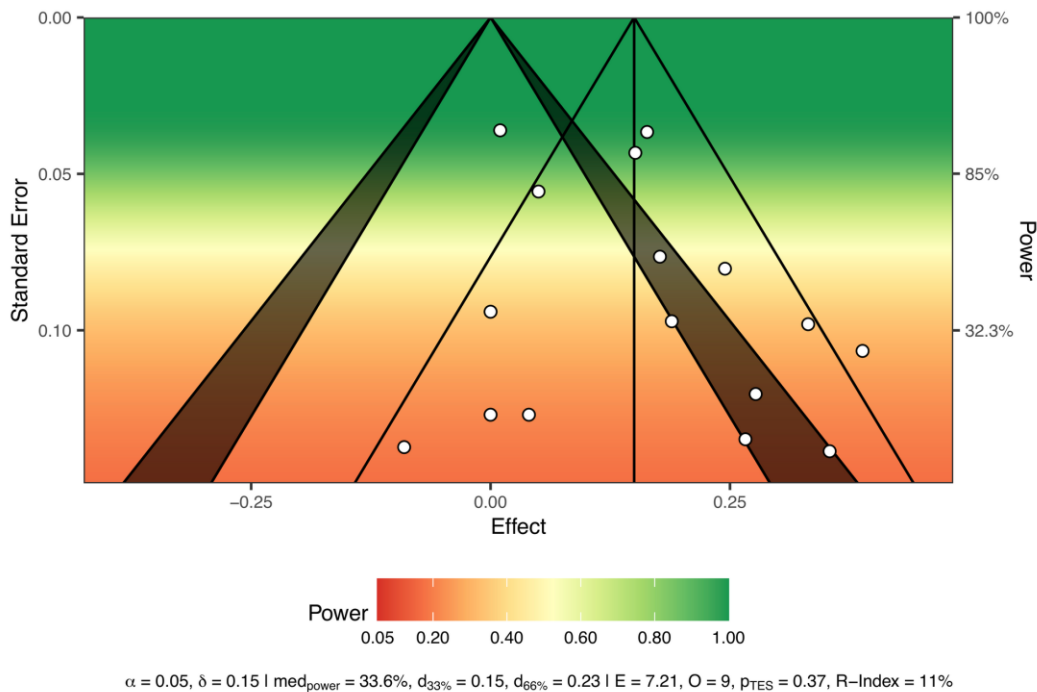
This analysis suggests that the publication bias adjusted estimate is 0.11, which is a little lower than the original estimate of 0.15.

So now we have two (out of our three) options for determining the "true" effect size. For the third option, let's assume that the smallest correlation coefficient we care about is 0.05.

We're now ready to calculate power and create a sunset plot. Let's begin by using the original effect size estimate. By default, this function uses the original effect size estimate so you don't need to specify it.


```
viz_sunset(res,
  contours = TRUE,
  power_contours = "continuous")
```

```
viz_sunset(res,
  contours = TRUE,
  power_contours = "continuous")
```



Let's walk through this plot.

Power is visualized by colour, with red used for "danger", as in, "these tests are very underpowered". The description at the bottom also displays the following:

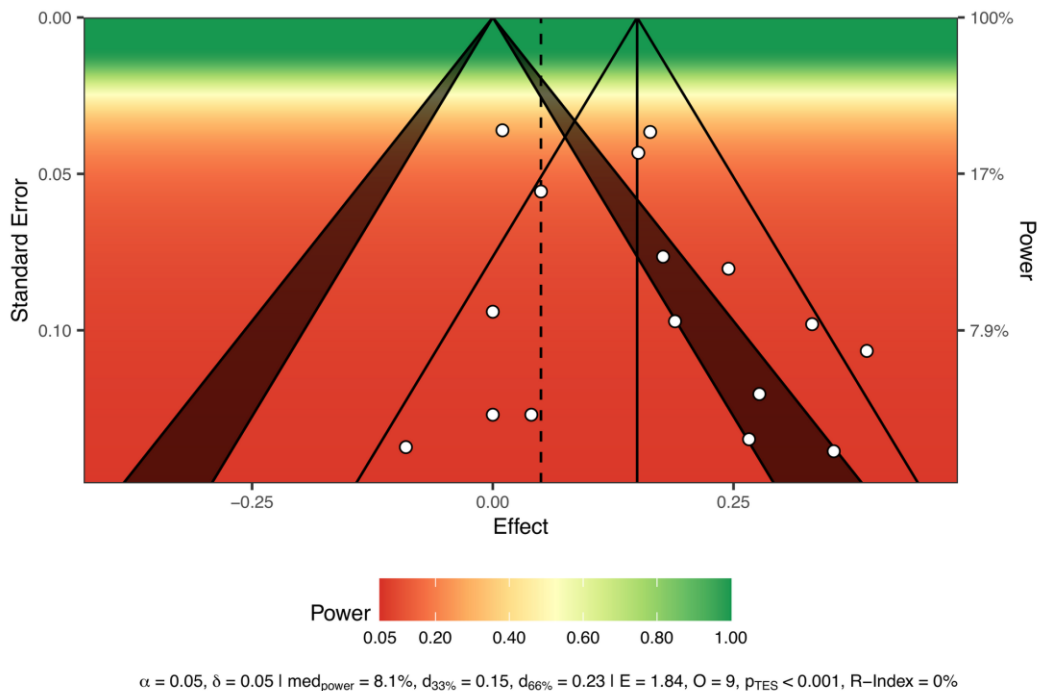
- The alpha (0.05)
- The true effect size (0.15)
- The median power of all the tests (33.6%)
- The true effect size needed for achieving 33% (0.15) and 66% (0.23) of median power
- Results from a test of excess significance (i.e., given the data we would expect 7.21 statistically significant studies and we observed 9, yielding a p-value of 0.37)
- An R-Index measure of 11%, suggesting that these studies have a low probability of replicating

To use our minimally interesting effect for the true effect size, we just need to tweak the code a little.

```
viz_sunset(res,
  contours = TRUE,
  true_effect = 0.05,
  power_contours = "continuous")
```

```
viz_sunset(res,
  contours = TRUE,
  true_effect = 0.05,
  power_contours = "continuous")
```

All tests reported from these studies are underpowered to detect this minimally interesting effect.



Finally, if you are interesting in extracting the exact power values, I have put together a small custom function.

```
s_power <- function(se, true_effect, sig_level) {
  (1 - stats::pnorm(stats::qnorm(1 - sig_level/2) *
    se, abs(true_effect), se)) +
  stats::pnorm(stats::qnorm(sig_level/2) *
    se, abs(true_effect), se)
}

s_power <- function(se, true_effect, sig_level) {
  (1 - stats::pnorm(stats::qnorm(1 - sig_level/2) *
    se, abs(true_effect), se)) +
  stats::pnorm(stats::qnorm(sig_level/2) *
    se, abs(true_effect), se)
}
```

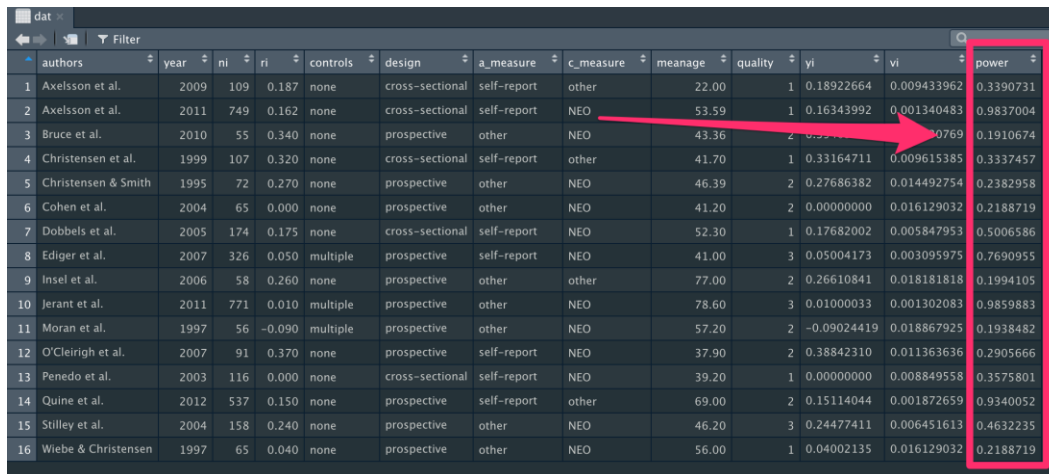
Let's use this function to add power calculations to each study using a true effect size of 0.15. We also need to specify the standard error for each effect, and significance level. Here,

we're using data from the funnel plot we made earlier to extract the standard error information.

```
dat$power <- s_power(se = funnel_plot$y,
  true_effect = 0.15,
  sig_level = 0.05)
```

```
dat$power <- s_power(se = funnel_plot$y,
  true_effect = 0.15,
  sig_level = 0.05)
```

Now we have the precise power estimates for each effect size at the end of our dataset.



	authors	year	ni	ri	controls	design	a_measure	c_measure	meanage	quality	yi	vi	power
1	Axelsson et al.	2009	109	0.187	none	cross-sectional	self-report	other	22.00	1	0.18922664	0.009433962	0.3390731
2	Axelsson et al.	2011	749	0.162	none	cross-sectional	self-report	NEO	53.59	1	0.16343992	0.001340483	0.9837004
3	Bruce et al.	2010	55	0.340	none	prospective	other	NEO	43.36	2	0.27686382	0.014492754	0.1910674
4	Christensen et al.	1999	107	0.320	none	cross-sectional	self-report	other	41.70	1	0.33164711	0.009615385	0.3337457
5	Christensen & Smith	1995	72	0.270	none	prospective	other	NEO	46.39	2	0.27686382	0.014492754	0.2382958
6	Cohen et al.	2004	65	0.000	none	prospective	other	NEO	41.20	2	0.00000000	0.016129032	0.2188719
7	Dobbels et al.	2005	174	0.175	none	cross-sectional	self-report	NEO	52.30	1	0.17682002	0.005847953	0.5006586
8	Ediger et al.	2007	326	0.050	multiple	prospective	self-report	NEO	41.00	3	0.05004173	0.003095975	0.7690955
9	Insel et al.	2006	58	0.260	none	prospective	other	other	77.00	2	0.26610841	0.018181818	0.1994105
10	Jerant et al.	2011	771	0.010	multiple	prospective	other	NEO	78.60	3	0.01000033	0.001302083	0.9859883
11	Moran et al.	1997	56	-0.090	multiple	prospective	other	NEO	57.20	2	-0.09024419	0.018867925	0.1938482
12	O'Leirigh et al.	2007	91	0.370	none	prospective	self-report	NEO	37.90	2	0.38842310	0.011363636	0.2905666
13	Penedo et al.	2003	116	0.000	none	cross-sectional	self-report	NEO	39.20	1	0.00000000	0.008849558	0.3575801
14	Quine et al.	2012	537	0.150	none	prospective	self-report	other	69.00	2	0.15114044	0.001872659	0.9340052
15	Stille et al.	2004	158	0.240	none	prospective	other	NEO	46.20	3	0.24477411	0.006451613	0.4632235
16	Wiebe & Christensen	1997	65	0.040	none	prospective	other	NEO	56.00	1	0.04002135	0.016129032	0.2188719

Here's the full script, if you'd like to run this all in one go.

```
library(metafor) #Load metafor
library(metaviz) #Load metaviz
library(puniform) #Load puniform

dat <- get(data(dat.molloy2014)) # Extract data

dat <- escalc(measure="ZCOR",
  ri=ri, ni=ni,
  data=dat) #Calculate effect sizes

res <- rma.uni(yi, vi, data=dat)

forest_plot <- forest(res)
forest_plot

funnel_plot <- funnel(res)
funnel_plot

funnel_contour <- funnel(res, level=c(90, 95, 99),
  shade=c("white", "orange", "red"),
  reflines=0, legend="topleft")

funnel_contour
```

```

puni_star(yi = dat$yi, vi = dat$vi,
          side = "right")

viz_sunset(res,
            contours = TRUE,
            power_contours = "continuous")

viz_sunset(res,
            contours = TRUE,
            true_effect = 0.05,
            power_contours = "continuous")

s_power <- function(se, true_effect, sig_level) {

  (1 - stats::pnorm(stats::qnorm(1 - sig_level/2) *
                    se, abs(true_effect), se)) +
  stats::pnorm(stats::qnorm(sig_level/2) *
                se, abs(true_effect), se)
}

dat$power <- s_power(se = funnel_plot$y,
                    true_effect = 0.15,
                    sig_level = 0.05)

library(metafor) #Load metafor
library(metaviz) #Load metaviz
library(puniform) #Load puniform

dat <- get(data(dat.molloy2014)) # Extract data

dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat) #Calculate effect sizes

res <- rma.uni(yi, vi, data=dat)

forest_plot <- forest(res)
forest_plot

funnel_plot <- funnel(res)
funnel_plot

funnel_contour <- funnel(res, level=c(90, 95, 99),
                          shade=c("white", "orange", "red"),
                          refline=0, legend="topleft")
funnel_contour

puni_star(yi = dat$yi, vi = dat$vi,
          side = "right")

viz_sunset(res,
            contours = TRUE,
            power_contours = "continuous")

```

```

viz_sunset(res,
            contours = TRUE,
            true_effect = 0.05,
            power_contours = "continuous")

s_power <- function(se, true_effect, sig_level) {

  (1 - stats::pnorm(stats::qnorm(1 - sig_level/2) *
                    se, abs(true_effect), se)) +
  stats::pnorm(stats::qnorm(sig_level/2) *
               se, abs(true_effect), se)
}

dat$power <- s_power(se = funnel_plot$y,
                    true_effect = 0.15,
                    sig_level = 0.05)

```

Oh my GOSH: Calculating all possible meta-analysis study combinations

Fuente: <https://www.dsquintana.blog/combinatorial-meta-analysis/>

There are several analytical decisions that need to be made when performing a meta-analysis. These decisions are not trivial, as they can influence meta-analysis outcomes and conclusions.

One of these decisions is deciding which studies to include in the meta-analysis. A single study, or group of studies, can have a disproportionate influence on the summary effect size.

Let's fire up R and have a closer look. Here, I'm going to load up the metafor package and use one of its built-in meta-analysis datasets. This meta-analysis dataset from Molloy and colleagues explores the relationship between conscientiousness and medication adherence.

```
metafor
```

Here's a basic meta-analysis, which extracts the data, calculates an effect size measure and effect size variance, and creates a forest plot for visualisation.

```
library(metafor) #Load metafor

dat <- get(data(dat.molloy2014)) # Extract data

dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat) #Calculate effect sizes

res <- rma.uni(yi, vi, data=dat)

forest_plot <- forest(res)

library(metafor) #Load metafor

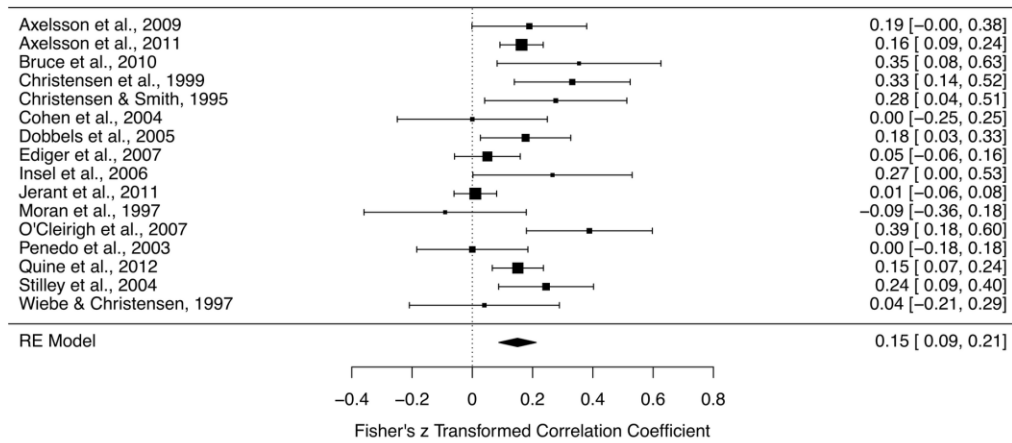
dat <- get(data(dat.molloy2014)) # Extract data

dat <- escalc(measure="ZCOR",
              ri=ri, ni=ni,
              data=dat) #Calculate effect sizes

res <- rma.uni(yi, vi, data=dat)

forest_plot <- forest(res)
```

Let's look at the forest plot that was generated from this script. The summary effect size was 0.15, and this was statistically significant, suggesting a modest relationship between conscientiousness and medication adherence.



To demonstrate the robustness of the result, it's good practice to perform a leave-one-out analysis. Like the name suggests, this analysis performs a series of meta-analyses leaving one of the studies out. So if you had 15 studies in your meta-analysis, you would have 15 meta-analyses.

Here is the script to run this:

```
l_one <- leavelout(res)
l_one <- leavelout(res)
```

And here are the results:

	estimate	se	zval	pval
Axelsson et al., 2009	0.1483	0.0336	4.4157	0.0000
Axelsson et al., 2011	0.1500	0.0354	4.2355	0.0000
Bruce et al., 2010	0.1418	0.0315	4.5064	0.0000
Christensen et al., 1999	0.1379	0.0308	4.4739	0.0000
Christensen & Smith, 1995	0.1441	0.0323	4.4550	0.0000
Cohen et al., 2004	0.1566	0.0326	4.8119	0.0000
Dobbels et al., 2005	0.1489	0.0342	4.3555	0.0000
Ediger et al., 2007	0.1599	0.0334	4.7841	0.0000
Insel et al., 2006	0.1455	0.0324	4.4863	0.0000
Jerant et al., 2011	0.1647	0.0291	5.6601	0.0000
Moran et al., 1997	0.1586	0.0317	5.0069	0.0000
O'Cleirigh et al., 2007	0.1353	0.0296	4.5647	0.0000
Penedo et al., 2003	0.1592	0.0324	4.9100	0.0000
Quine et al., 2012	0.1514	0.0353	4.2858	0.0000
Stilley et al., 2004	0.1432	0.0330	4.3432	0.0000
Wiebe & Christensen, 1997	0.1552	0.0329	4.7203	0.0000

The summary effect sizes for meta-analyses that removed a single study ranged from 0.14 to 0.16. Thus, removing a single study did not have much influence on the primary summary effect size, which as 0.15.

This analysis is quite informative, but perhaps there's a distinct subgroup of studies influencing the result, rather than a single study.

So let's what's called a combinatorial meta-analysis, which performs a series of meta-analyses based on all possible combinations of studies.

The number of analyses to be conducted can be calculated using the formula $2^k - 1$, with k representing the number of studies. So for a meta-analysis with 16 studies, you will perform 65535 meta-analyses, that is $2^{16} - 1$.

$2^k - 1$

$2^{16} - 1$

To visualise this enormous number of summary effect size estimates, we'll put together a Graphical Display of Study Heterogeneity (GOSH) plot, using the following script, but before running the script, I should warn you that this could take some time to create 65535 models (it took about 10 minutes on my 2015 MacBook Pro).

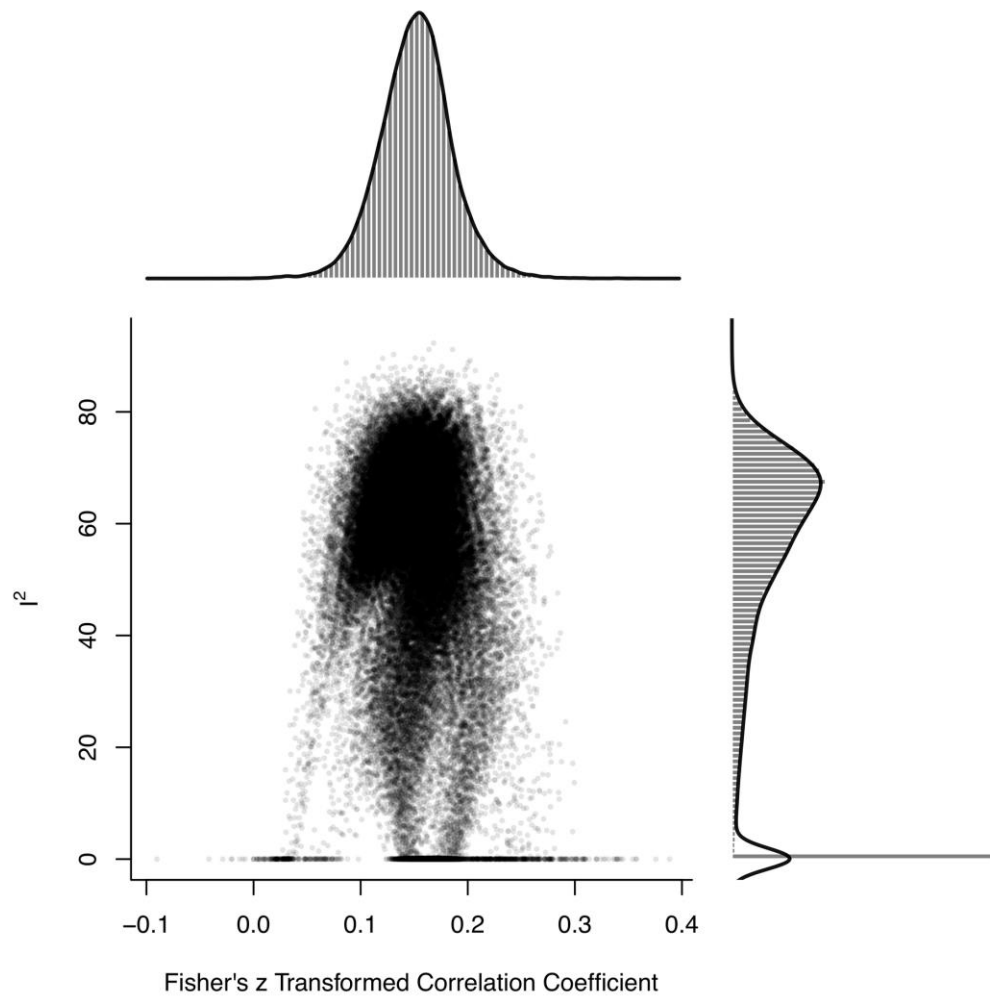
```
gp <- gosh(res)
```

```
g_plot <- plot(gp, breaks = 100)
```



```
g_plot
gp <- gosh(res)
g_plot <- plot(gp, breaks = 100)
g_plot
```

Here's a visualisation of the summary effects of 65535 models (horizontal axis), plotted against the model's heterogeneity (vertical axis).



By extracting the list of effect sizes, it's straightforward to calculate the mean effect size, which is 0.15.

```
mgosh <- gp$res
mean_est <- mean(mgosh$estimate, na.rm = TRUE)
```

```
mgosh <- gp$res
```

```
mean_est <- mean(mgosh$estimate, na.rm = TRUE)
```

The effect sizes range from about 0.05 to 0.25, demonstrating how the choice of included studies influences the summary effect size estimate.

Now let's see what happens when we add in some artificial outliers, using this script:

```
dat_mo <- get(data(dat.molloy2014))
```

```
dat_mo <- escalc(measure="ZCOR",  
                ri=ri, ni=ni, data=dat_m)
```

```
dat_mo[12,11] = 0.7
```

```
dat_mo[15,11] = 0.6
```

```
res_mo <- rma.uni(yi, vi, data=dat_mo)
```

```
forest(res_mo)
```

```
dat_mo <- get(data(dat.molloy2014))
```

```
dat_mo <- escalc(measure="ZCOR",  
                ri=ri, ni=ni, data=dat_m)
```

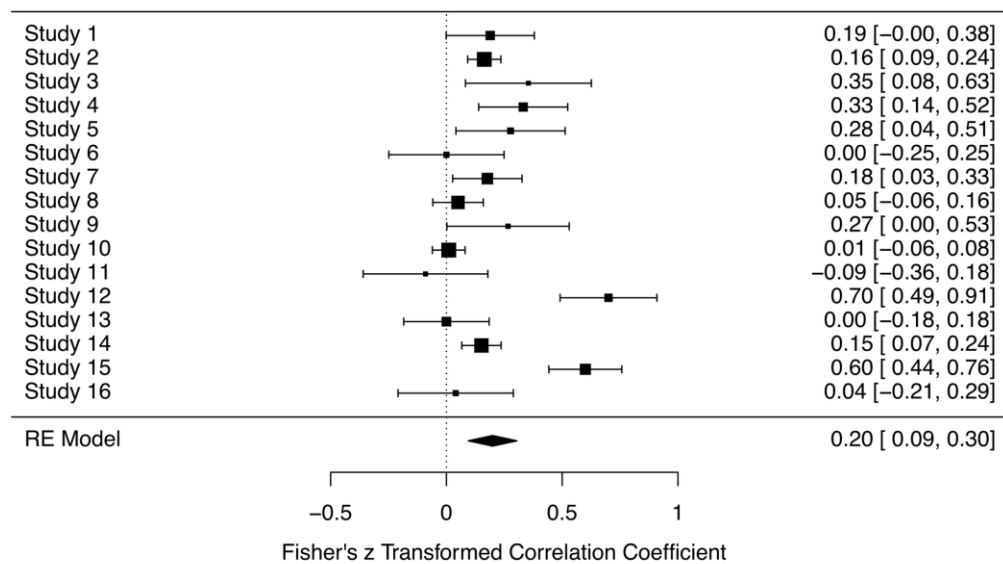
```
dat_mo[12,11] = 0.7
```

```
dat_mo[15,11] = 0.6
```

```
res_mo <- rma.uni(yi, vi, data=dat_mo)
```

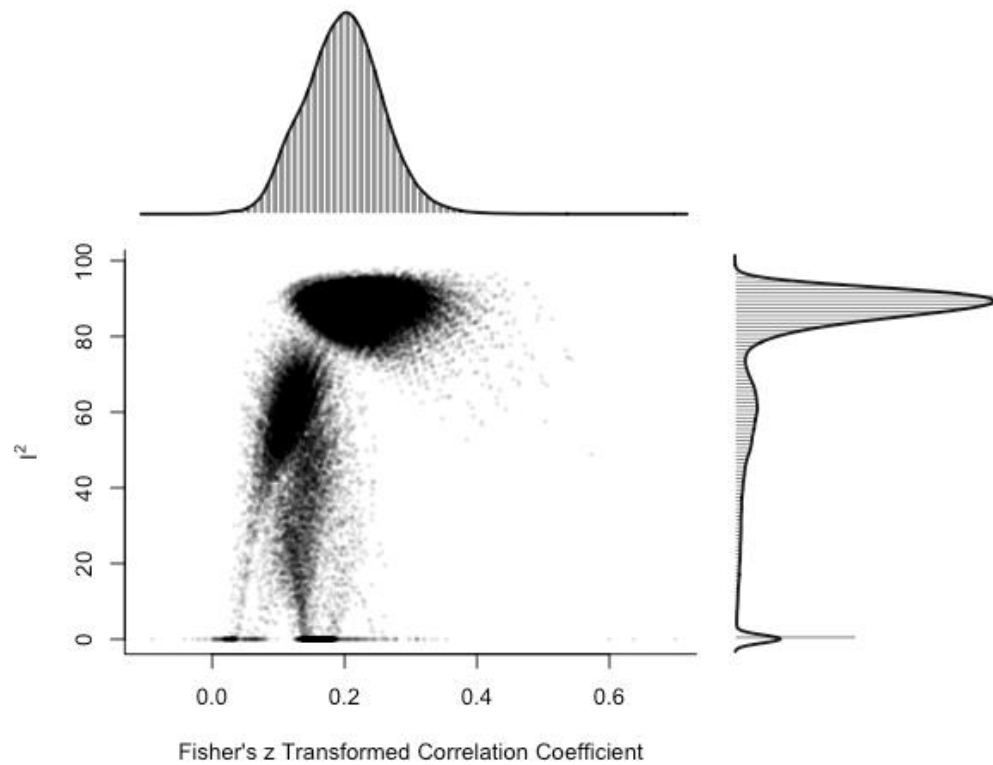
```
forest(res_mo)
```

You can see the introduced outliers in the forest plot (studies 12 and 15)



Now let's run the combinatorial meta-analysis and create a GOSH plot:

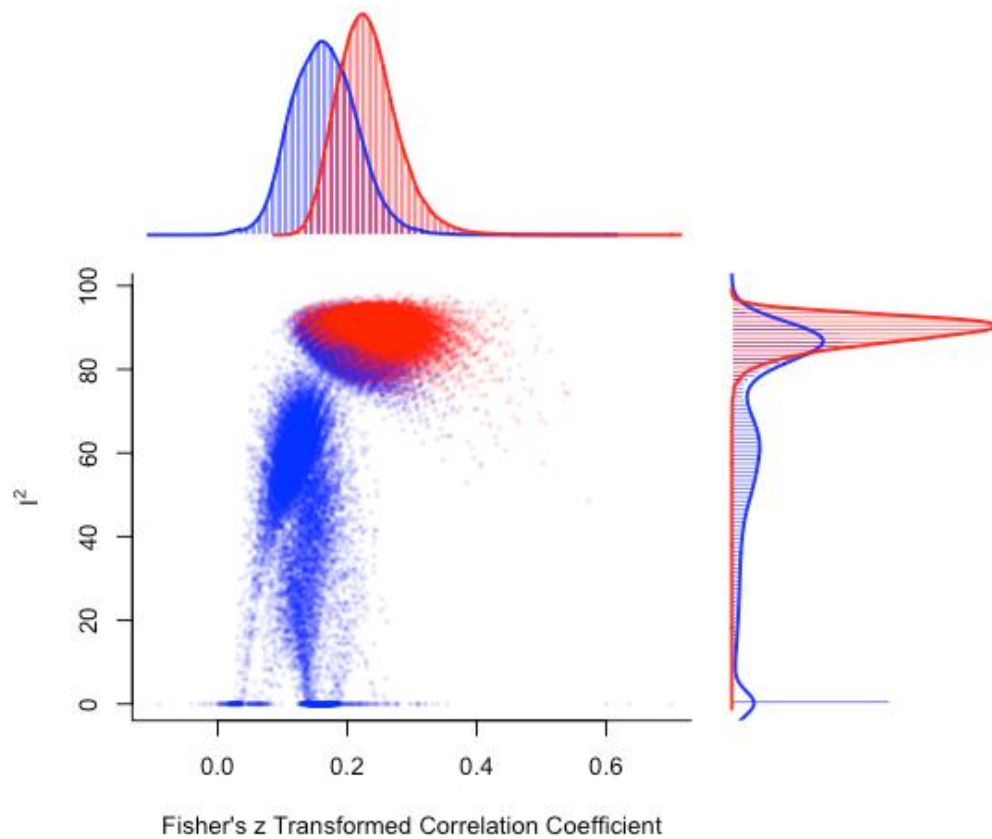
```
gp_mo <- gosh(res_mo)
plot(gp_mo, breaks = 100)
gp_mo <- gosh(res_mo)
plot(gp_mo, breaks = 100)
```



There is a clear cluster of large effect sizes with high heterogeneity in the top right of the plot. We can specifically highlight the meta-analysis models that included the potential outlier studies:

```
plot(gp_mo, out=12, breaks = 100)
```

```
plot(gp_mo, out=12, breaks = 100)
```



There's a noticeable shift in the distribution of meta-analysis model summary effect size estimates that include study 12.

Ultimately, combinatorial meta-analysis paired with GOSH plots demonstrate how your analysis choices influence your meta-analysis outcomes and conclusions.

How to make a free website in R

Fuente: <https://www.dsquintana.blog/free-website-in-r-easy/>

UPDATE: These instructions are now outdated. See my 2020 guide instead.

If you're an academic you need a website so that people can easily find info about your research and publications. You will miss opportunities, like speaking invitations and collaborations, if scholars can't find your information online.

This tutorial will walk you through how to make your own website for free using the blogdown package in RStudio.

blogdown

There are some great point-and-click website creation services available, like Squarespace, you need to pay a fee of about \$10 a month and the templates are not well-suited for academic webpages.

Alternatively, blogdown and RStudio is free and can integrate with the Hugo framework, which provides several website templates. It also uses Markdown, which is a straightforward markup language. So let's get started!

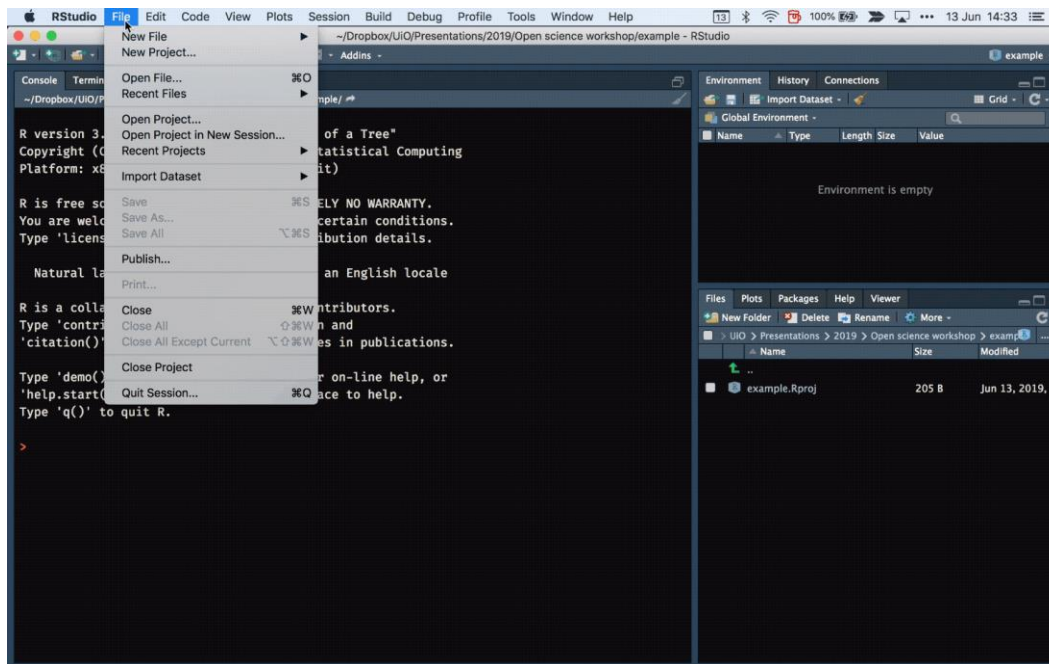
blogdown

Install blogdown in R using this command `install.packages("blogdown")`

blogdown

```
install.packages("blogdown")
```

Then start a new project, entering "gcushen/hugo-academic" as the Hugo theme. Keep the other options ticked. This will download all the necessary files.



Now, load blogdown using this command `library(blogdown)` and then install the "hugo" framework, which is the backbone of your website `blogdown::install_hugo(force = TRUE)`

```
library(blogdown)
```

```
blogdown::install_hugo(force = TRUE)
```

Then build the template site using this command `blogdown::serve_site()`

```
blogdown::serve_site()
```

The viewer window will render a mobile version of your site, but you can also see a desktop version in your browser.

So now you can see what your website will look like. All you have to do is to change the information in your website files. Let's start with the title of your website, which can be changed in the `config.toml` file.

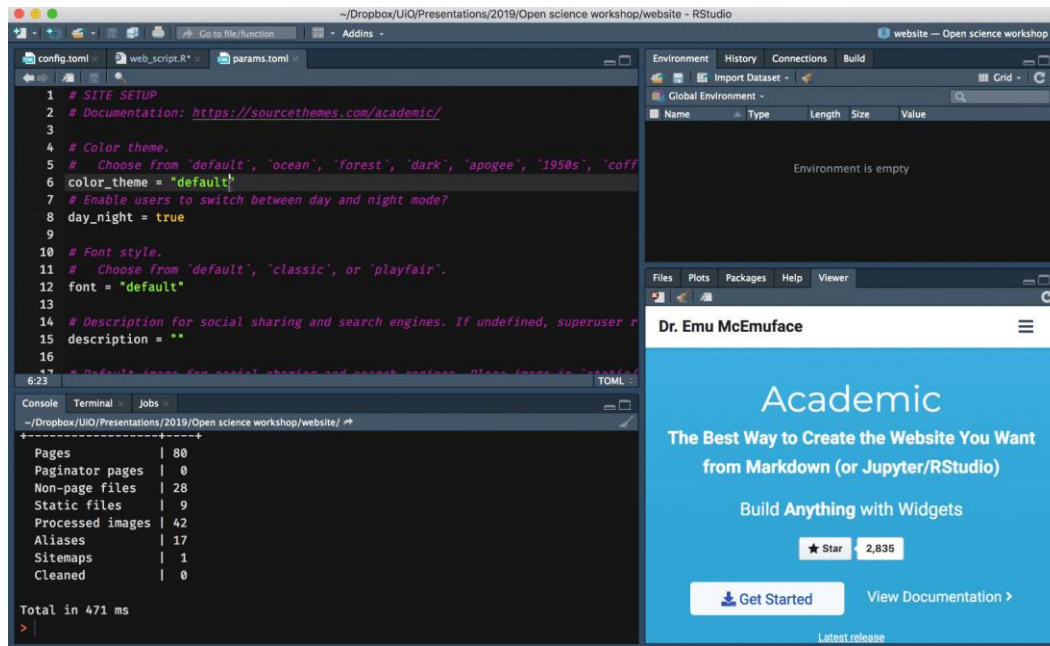
```
config.toml
```

To edit this file, and the others that are referred to in this guide, just select it in the RStudio file browser.

Of course, you can edit these files in a text editor, but it's much easier to edit these files within RStudio.

In the `config/_default/params.toml` file you can change the color theme and font style of your website, so have a play around with these options.

config/_default/params.toml

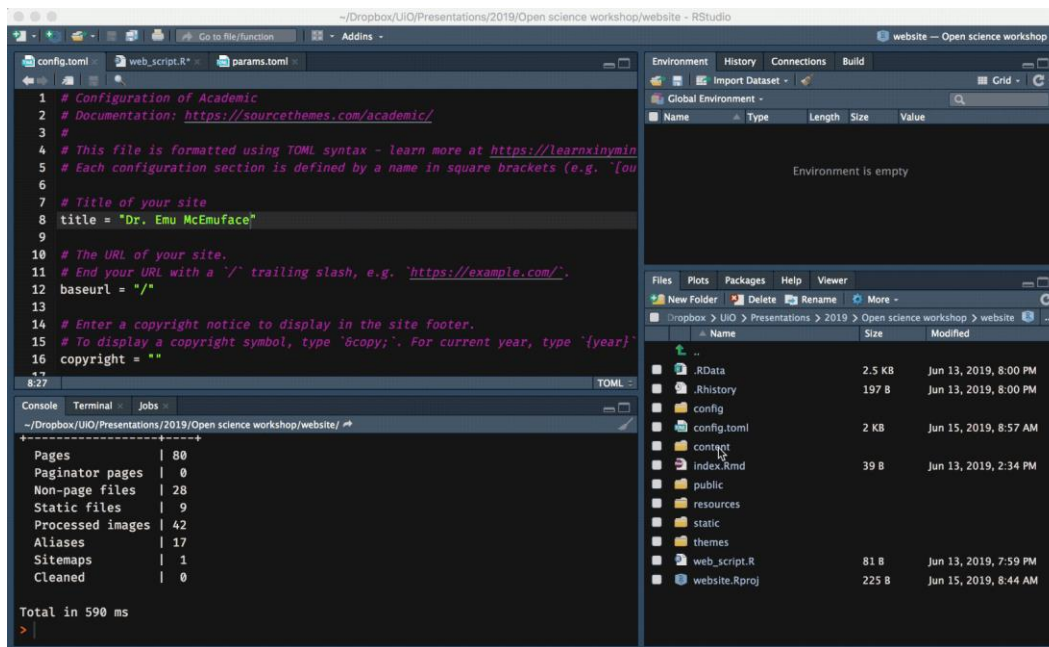


Your main homepage is made up of a set of widgets, which you can customize or remove entirely. For example, let's say we want to remove the big header image, called the "hero" widget. Let's open up the `content/home/hero.md` file and change `active = true` to `active = false`

`content/home/hero.md`

`active = true`

`active = false`



Let's now update the profile photo. Just save your profile in the content/authors/admin folder, calling the file avatar.jpg. This will automatically update your profile picture.

```
content/authors/admin
```

```
avatar.jpg
```

Edit your biography details (e.g, position, affiliation, education details) in the content/authors/admin/_index.md file. In this file you can also add your social media details and a link to your Google Scholar profile page.

```
content/authors/admin/_index.md
```

To edit your contact details, navigate to config/_default/params.toml and scroll down to the "Contact Widget setup" section.

```
config/_default/params.toml
```

To add a PDF of your CV, copy your CV to static/files/cv.pdf and uncomment (i.e., remove the # signs) that prefix the CV section lines of the file.

```
static/files/cv.pdf
```

```
#
```

To see how your updates look on your website, save the file in Rstudio, and then run the blogdown:::serve_site() command.

```
blogdown:::serve_site()
```

These markdown (.md) files have detailed comments on what each bit code does. If you're still not sure what something does, just edit your script and see what happens. You can always undo your changes.

To edit your publications, go to the files found in the content/publication/ folder. Each publication has a dedicated folder. To include your first publication, navigate to the content/publication/journal-article folder, and open up the index.md file.

```
content/publication/  
content/publication/journal-article  
index.md
```

Edit your publication's details and then save your file. You might want to rename your folder to make it easier to identify. For add more publications, simply copy and paste the example folder and edit accordingly.

To include an image to be associated with your paper (e.g., a figure from your paper), just add it to your publication's folder and call it featured.jpg.

```
featured.jpg
```

To change what appears in your website's header menu, edit the config/_default/menus.toml file. Just edit or delete sections as you wish. If you would like to add your CV to the menu, just add a menu item, which you can call CV (name = "CV"), which includes url = "files/cv.pdf".

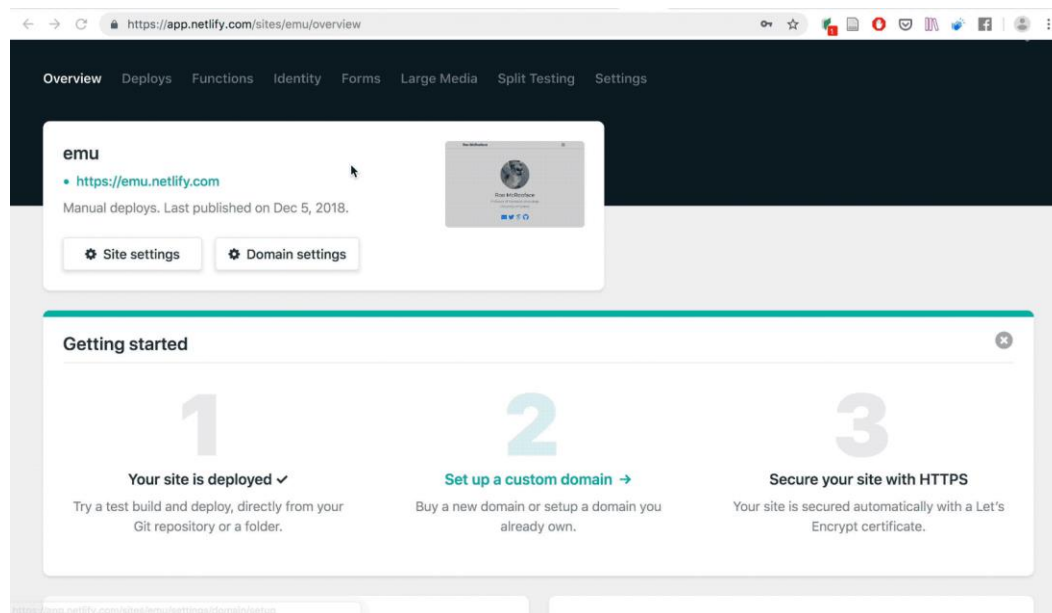
```
config/_default/menus.toml  
  
name = "CV"  
  
url = "files/cv.pdf"
```

When you're done with your edits and you're happy with your page, it's time to put it online. The easiest way to do this is via Netlify.

One way to upload your website and keep it updated is by hosting your files on Github. If you're going to be updating your site regularly, this is probably the best way to go, but this is the topic of another tutorial post.

But if you want to get set up now, just drag the public folder into Netlify within the "deploys" section of your admin settings. It should take about 5-10 seconds for your site to go live.

```
public
```



You can edit your domain name, which will include "netlify.com" at the end of your address. If you would like a cleaner domain name, you'll can purchase one for the cost of a few coffees per year. I recommend Namecheap, who is currently offering free domain names for university students.

Here's the website we just made. For a completed website, here's mine with a custom domain name.

If you've made your own website using this tutorial, let me know on Twitter!

Labeling boxplot outliers in R

Fuente: <https://www.dsquintana.blog/labeling-boxplot-outliers/>

Boxplots provide a useful visualization of the distribution of your data. Typically, boxplots show the median, first quartile, third quartile, maximum datapoint, and minimum datapoint for a dataset. Many boxplots also visualize outliers, however, they don't indicate at glance which participant or datapoint is your outlier.

Labeling your boxplot outliers is straightforward using the `ggstatsplot` package, here's a quick tutorial on how to do this.

```
ggstatsplot
```

Let's first install and load our required packages. First, we'll need the `tidyverse` package as it comes with a dataset of Star Wars character attributes that I'll be using and we'll need to clean a dataset a little. Second, we're going to load the `ggstatsplot` to construct boxplots and tag outliers.

```
tidyverse
```

```
ggstatsplot
```

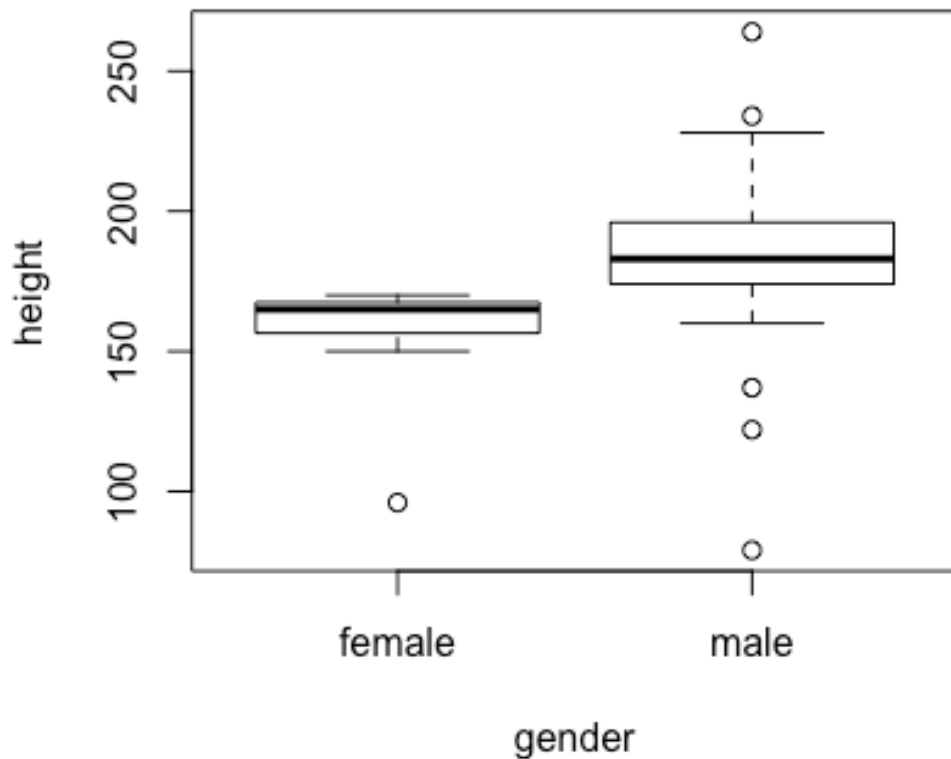
In humans, males are typically taller than females, but what about males and females in the Star Wars universe, which is inhabited by thousands of different species? Let's take a look in our dataset.

Let's clean up our dataset for the purposes of this demonstration by only including males and females as there's a single hermaphrodite in the dataset—it's Jabba the Hutt, if you're wondering.

To clean our dataset, we're using the "filter" function from the `dplyr` package, which comes with the `tidyverse` package. We'll also construct a standard boxplot using base R.

```
dplyr
```

Here's our base R boxplot, which has identified one outlier in the female group, and five outliers in the male group—but who are these outliers?



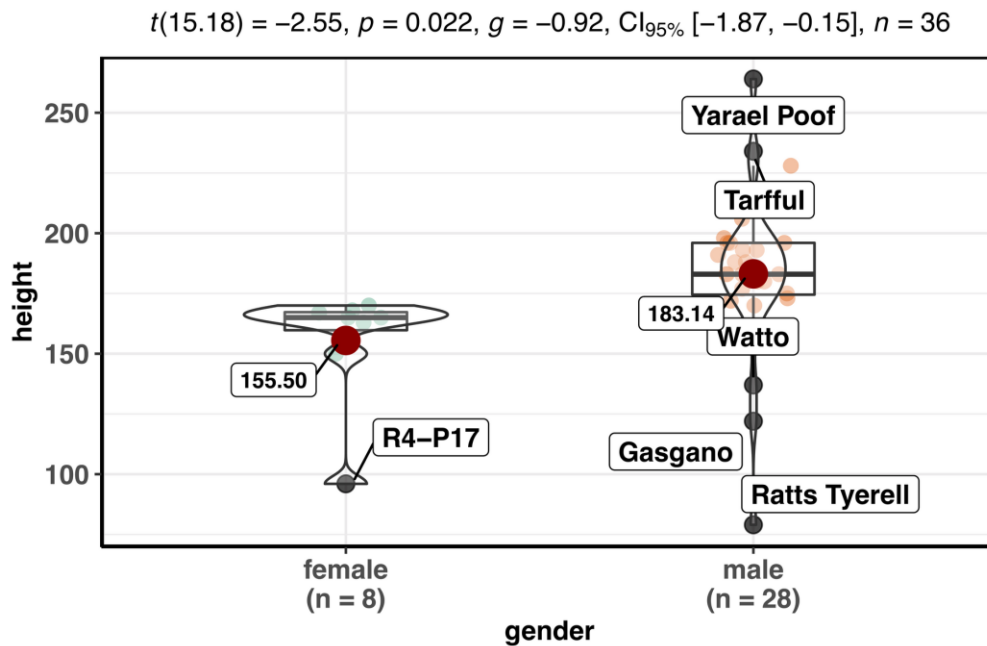
We can identify and label these outliers by using the `ggbetweenstats` function in the `ggstatsplot` package.

```
ggstatsplot
```

To label outliers, we're specifying the `outlier.tagging` argument as "TRUE" and we're specifying which variable to use to label each outlier with the `outlier.label` argument.

Here's our plot with labeled outliers. A great feature of the `ggstatsplot` package is that it also reports the result of the statistical test comparing these two groups at the top of the plot. Males were significantly taller than females in this dataset.

```
ggstatsplot
```



By default, the ggstatsplot package also identifies and labels the group means (the red dots), which is typically of interest but seldom included in conventional boxplots.

`ggstatsplot`

Here's the full R script for this tutorial, all in one place.

Equivalence testing: What to do with a non-significant result

Fuente: <https://www.dsquintana.blog/equivalence-testing-non-significant-result-p-value/>

I recently did a series of talks at the "Psychology and the Brain" conference in Germany. I managed to film one these talks, which was on how to better understand non-significant results, for a livestream. Here's the replay:

LIVE PRESENTATION: Using equivalence testing to better understand non-significant results
<https://t.co/h56MffTscs>— Dan Quintana (@dsquintana) June 20, 2019

LIVE PRESENTATION: Using equivalence testing to better understand non-significant results <https://t.co/h56MffTscs>

Here's a link to the slides that I used.

Reproducible and future-proof analysis in R

Fuente: <https://www.dsquintana.blog/reproducible-and-future-proof-analysis-in-r-open-science/>

Here's a screencast I recently made on how to make a reproducible version of your R analysis.

This is incredibly easy to set up, so if you would like to easily share your analyses you should check this out.

How to visualise research trends in R

Fuente: <https://www.dsquintana.blog/how-to-visualise-research-trends-in-r/>

If you've ever wanted to visualise research trends for a paper introduction or grant application, the {europepmc} R package makes this really easy.

Here's the code used in this screencast.

